

8. Análise de Requisitos

“Noite, a amada. Noite, quando palavras se dissolvem e as coisas tornam-se vivas. Quando a destrutiva análise diária está completa, e tudo que é realmente importante se torna inteiro e razoável novamente. Quando o homem junta outra vez o seu ser fragmentado e cresce com a calma de uma árvore.”

Antoine de Saint-Exupéry Este capítulo descreve a análise de requisitos, as atividades que a compõem e os objetivos que se pretende alcançar com a sua realização, tendo em perspectiva a sua inter-relação com os demais processos da Engenharia de Requisitos. Os insumos para a análise são os resultados da elicitação. Os produtos da análise são os requisitos da solução e da transição, geralmente na forma de especificações com requisitos funcionais e não funcionais prontos para encaminhar para empacotamento em unidades coesas para o projeto do software. Também são resultados da análise hiatos de informação ou oportunidades de racionalização que retroalimentam novas atividades de elicitação; conflitos não passíveis de resolução com as informações disponíveis para análise são direcionados para a gerência de requisitos.

A análise de requisitos é apresentada não só com o objetivo de organizar requisitos, mas também como instrumento para descoberta de requisitos implícitos, ocultos e cuja descoberta tardia pode inviabilizar o projeto e tornar nulos os resultados para os investimentos já realizados.

8.1. Um problema: a descoberta tardia de que ainda falta muito

O produto de software não atende às necessidades de negócio. As partes interessadas descobrem isso apenas quando do teste de aceitação ou mesmo na tentativa de transição para o ambiente real (produção). Essa é uma das experiências mais frustrantes para todos os envolvidos. Isso também traz graves consequências para o projeto, que pode ser cancelado por não mais se justificar; e para o negócio, que despendeu recursos valiosos para permanecer com sua necessidade não atendida.

Projetos apresentam um fenômeno mais comum do que se pode imaginar: estar próximo à metade do caminho quando se acredita que esteja concluído (ou quase).

O fenômeno não é motivado necessariamente por uma mudança no escopo original, como poderia se pensar em um primeiro momento. O escopo original deve ser definido em termos de áreas funcionais (ou mesmo organizações) afetadas e macroprocessos de negócio que limitam a abrangência da solução nesse nível mais alto. Essa declaração de escopo não deve limitar quais sejam especificamente os requisitos da solução ou de transição. Por isso, a descoberta desses requisitos mais específicos é uma revelação própria do esforço de desenvolvimento de requisitos e não deve ser vista como uma mudança ou causa que explique ou justifique o fenômeno descrito.

Parece então um paradoxo, pela contradição implícita. O paradoxo se desfaz quando se esclarece qual o marco utilizado para avaliar os 50% concluídos e qual o outro marco para avaliar os 100% (ou, mais frequentemente, 90%). O último considera o atendimento ao escopo identificado, enquanto o outro considera um escopo necessário para atender plenamente ao propósito para o qual o projeto foi iniciado. A Figura 8.1 ilustra esse cenário.

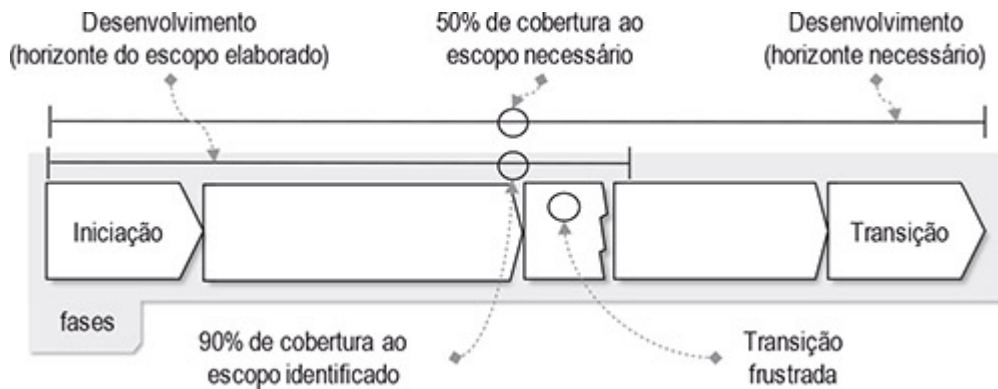


Figura 8.1. A dinâmica da crença em 90% concluídos quando ainda está bem distante do término.

A análise deve antecipar a descoberta do horizonte representado pelos 100% (ou ao menos os 80% mais críticos desse horizonte em um primeiro momento). Com isso, encontra uma bela oportunidade de evitar que o fenômeno descrito se multiplique. Deve-se ressaltar que a análise é parte da solução porque há ações necessárias que dependem de decisões em outros âmbitos, como, por exemplo, ciclos de *feedback* mais curtos entre o projeto e os usuários e com resultados intermediários mais tangíveis.

8.2. Visão geral da análise de requisitos

Se a elicitação de requisitos descobre as peças do quebra-cabeças, então a análise de requisitos procura montá-lo. O objetivo da análise de requisitos é aumentar o entendimento atual da informação, completá-la e melhorá-la. As informações descobertas com a elicitação das necessidades de negócio e dos requisitos das partes interessadas são seus insumos. A Figura 8.2 ilustra essa visão preliminar da análise de requisitos.

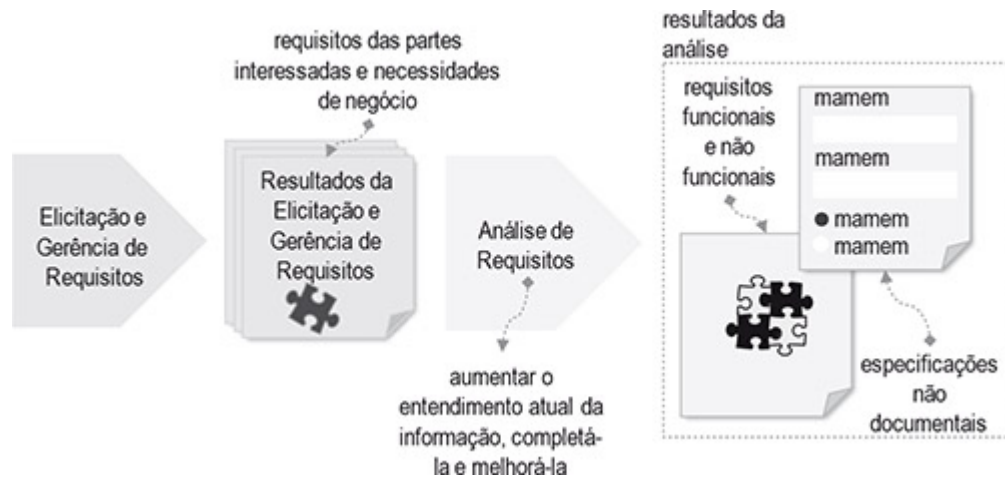


Figura 8.2. Uma visão resumida da análise de requisitos.

Essa visão preliminar é uma simplificação, pois coloca a análise sucedendo a elicitação, como que em um fluxo e sem *feedback*. Não costuma ser assim porque os produtos da elicitação são potencialmente omissos; ambíguos; redundantes; equivocados; ou incluem conflitos entre si.

Uma representação mais completa da análise deve incluir como parte de seus produtos pautas para novas sessões de elicitação a partir da descoberta dessas necessidades de informação ou decisão. Ela deve também posicionar a relação entre a análise e a elicitação como bidirecional. A Figura 8.3 faz isso.

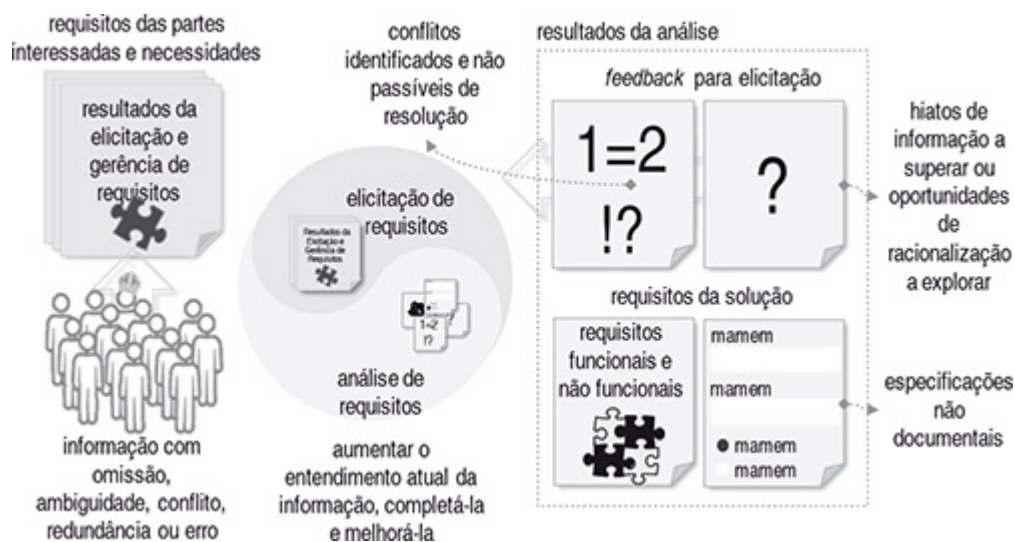


Figura 8.3. Dinâmica da interação entre elicitação e análise, onde os produtos de uma são insumos para a outra.

8.2.1. O que é análise de requisitos?

A análise de requisitos transforma a informação presente nos requisitos das partes interessadas e nas necessidades de negócio em diferentes processos expostos na Figura 8.4 de: ➤

exame da informação;



decomposição da informação; e ➤
síntese da informação.

O produto primário da análise de requisitos é a inteligência de negócio registrada em documentos e modelos com as especificações de requisitos funcionais e não funcionais para o software. Porém, a inexistência de especificações em documentos não implica na ausência de atividades de análise.

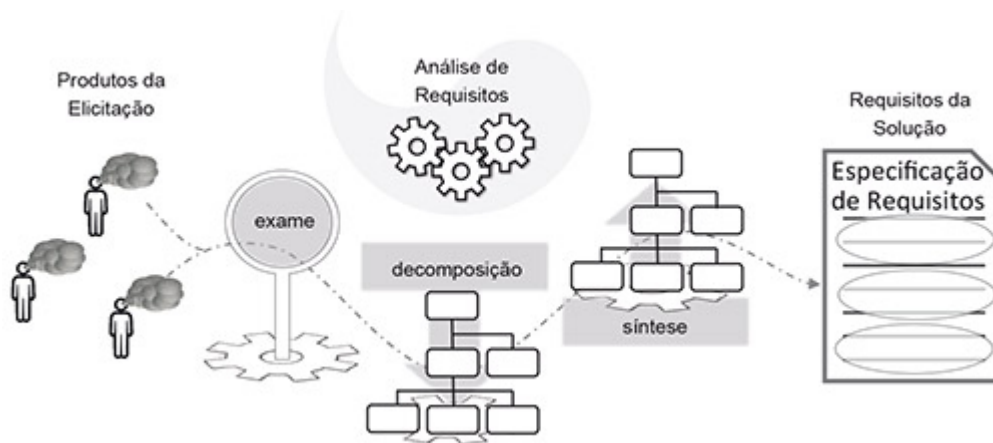


Figura 8.4. As habilidades básicas na análise de requisitos.

8.2.2. Por que análise de requisitos?

Há um grupo com 14 partes interessadas que fornecem requisitos com diferentes responsabilidades e autoridades nesse sentido. É razoável encaminhar para a implementação esses requisitos, com seus conflitos e oportunidades de racionalização ainda a serem revelados?

Bem, tudo é possível. Mas a qual custo com retrabalho que poderia ser evitado? Ainda assim, nesse cenário, haveria trabalho de análise de requisitos; contudo, pagando-se mais e esperando-se mais tempo. Algum retrabalho é inevitável em um processo de descoberta; é válido até questionar o uso do termo “retrabalho”. Não haver organização e disciplina em atividades de análise promove retrabalho além do necessário para o desenvolvimento dos requisitos.

A análise de requisitos tem por objetivo aumentar o entendimento atual da informação, completá-la e melhorá-la ao identificar omissão, ambiguidade, conflito, redundância ou erro nos requisitos.

O meio pelo qual a análise de requisitos cumpre esse objetivo é através da formulação de questionamentos a serem usados como importantes insumos para novas atividades de elicitação. Daí, pode-se dizer que há um fluxo de retroalimentação entre as atividades de elicitação e análise.

8.2.3. Como realizar a análise de requisitos?

A análise de requisitos é olhar mais de perto especificamente cada pedaço de informação revelado na elicitação e sobre como eles se relacionam entre si em conjunto. Envolve elaborar a informação de maneira progressiva e iterativa para descrever: ➤

O motivo para a mudança: que é a necessidade de negócio, o domínio do problema e um escopo inicial em alto nível.



A abrangência da solução (e da transição): que é o desenvolvimento de um escopo em termos específicos de seus requisitos funcionais e não funcionais.



A profundidade da solução (e da transição): que é o detalhamento com o passo a passo sobre esses requisitos no escopo.

Para tanto, a análise deve prover estrutura aos requisitos e às informações relacionadas. Ela consiste de diferentes tipos de atividade, a seguir relacionados, para o desenvolvimento da informação em direção a um conhecimento que pode ser aplicado para atingir os objetivos do negócio: ➤

Organizar a partir de exame, decomposição e síntese.



Modelar e usar modelos para refinar a informação.



Especificar para documentar os requisitos.



Verificar se o processo e os produtos observam boas práticas.



Validar se a solução satisfaz o cliente.

8.3. Organizar a partir do exame, decomposição e síntese

A elicitação de requisitos junto às partes interessadas resulta em documentos isolados (ou registros não documentais) com memórias de levantamento confirmadas, que devem ser examinadas durante a análise de requisitos. O analista deve considerar o maior contexto da mudança em que o software se insere. Nesse contexto, são tomadas decisões que: ➤

mantêm o fluxo operacional existente; e ➤

inovam em relação a um fluxo operacional existente.

Fluxo operacional deve ser entendido como a forma pela qual são realizadas as tarefas nos procedimentos formais e nas práticas estabelecidas na organização. A palavra-chave é **tarefa**.

8.3.1. Exame para identificar ou descrever tarefas

Os requisitos das partes interessadas são difusos, fragmentados e escondem muitas questões em aberto. Com o objetivo de facilitar a descoberta e a resolução dessas questões, definiu-se o conceito de nível de granularidade associado aos objetivos de um requisito.

Para ilustrar a descoberta dessas questões, considere dois requisitos levantados junto às partes interessadas para “Controlar entrada e saída de pessoas e veículos nas instalações portuárias” e “Validar o CPF do visitante na Receita Federal”.

O exame em discussão busca identificar requisitos como esses, que agregam vários objetivos do usuário. O trabalho ou qualquer tipo de comportamento que se deseja descrever, como manifestar que curtiu uma foto, pode ser descrito em uma infinidade de níveis de abstração. O mais material e que costuma ser compreendido até por trabalhadores braçais menos acostumados com conceitos abstratos é o da tarefa. Organizar os requisitos deve ter por objetivo nivelá-los para que possam ser tratados de maneira mais concreta e menos abstrata no nível de objetivo do usuário.

Por exemplo, um terceiro requisito “Listar todos os incidentes cadastrados em um período, segregados por área portuária” permite depreender especificamente uma tarefa; já os outros dois citados, não. Esses outros escondem questões como: ➤

Quais são as tarefas cuja realização deve ser alocada ao software em desenvolvimento para que se atinja o objetivo de controlar entrada e saída de pessoas e veículos nas instalações portuárias?

□

Quais são as tarefas em que se deve validar o CPF do visitante junto à Receita Federal?

Sobre a identificação dessas tarefas, sobre as respostas às perguntas citadas, qualquer projeto que inclui o desenvolvimento de software deve organizar os requisitos como comportamento que se deseja transferir para o software a partir de tarefas nas quais ele passará a se inserir.

Destaca-se que dificilmente se tem sucesso em identificar de maneira completa o escopo nesses termos já em momentos iniciais de um projeto.

8.3.1.1. Tarefas já existentes no fluxo operacional

O desenvolvimento do escopo nesses termos exige que decisões sejam tomadas pelas partes interessadas sobre quais tarefas devem ser absorvidas pelo software e em que grau ele deve absorvê-las. Trata-se de decisões que devem ser tomadas conforme a melhor adequação às necessidades de negócio e restrições do domínio do problema.

São decisões que implicam em:



Identificar tarefas: antes realizadas por um intermediário – que devem passar a ser realizadas diretamente por quem as origina ou para quem os seus produtos de informação se destinam. Por exemplo, quantas tarefas – antes realizadas por um caixa bancário – passam a ser feitas diretamente pelo cliente do banco quando movimenta seus recursos por meio de software (autoatendimento ou *internet banking*) para esse fim? Quantas tarefas – antes realizadas por um atendente em uma cooperativa de táxis – passam a ser executadas pelo próprio passageiro usando um aplicativo em seu celular?



Consolidar tarefas: até então definidas como tarefas diferentes. Com as facilidades oferecidas pela tecnologia da informação e comunicação, a separação em duas ou mais tarefas como reflexo de diferentes responsabilidades pode se revelar não mais necessária. Por exemplo, como parte da liberação de crédito prevista em um contrato de financiamento, há a atualização da planilha financeira com o registro do evento; novo cálculo para determinar valor de parcelas e encargos a vencer; e sua atualização em outras linhas da planilha financeira. Antes do projeto, havia duas tarefas diferentes no procedimento operacional abordado. Com a possibilidade de

transferir o processamento descrito para o software, as duas podem ser consolidadas em uma única.



Transferir parte do processamento de tarefas: ainda que mantenham a dependência de intermediários – o software, de maneira a garantir o correto intercâmbio de informações, e a observação das regras de negócios que se aplicam. Por exemplo, a avaliação de condições para determinar quais se aplicam ao informar um pedido que antes era de responsabilidade de uma pessoa e que passa a ser realizado pelo software, e, portanto, a tarefa em que isso acontece deve ser incluída como parte do escopo na análise dos requisitos.

8.3.1.2. Inovação no fluxo operacional

Enquanto os três casos expostos antes se referem a tarefas já presentes no fluxo operacional, há comportamento – proativo ou em resposta a um evento externo – que se estabelece como requisito para o software e que corresponde à inovação. São novas tarefas que talvez nem existissem se não houvesse a tecnologia da informação para suportá-las.

Por exemplo, originalmente, quando de uma renegociação das condições de um contrato de financiamento, era necessário celebrar um novo contrato estabelecendo os novos termos e condições aplicáveis e, então, com os recursos advindos desse novo contrato, quitar o anterior. O motivo dessa organização do trabalho é a complexidade associada a um procedimento único considerando as limitações de um indivíduo. Uma vez que essa complexidade esteja alocada como um requisito para o software, a situação é diferente; é possível criar uma tarefa única para esse fim.

8.3.1.3. Diferentes objetivos de informação

Sobre o nível de detalhe dos requisitos, dependendo do momento em que se encontra o projeto, a organização dos requisitos pode buscar descrever o escopo nos termos apresentados ou descrever o comportamento de cada tarefa em particular.

8.3.1.3.1. Descrever o escopo em termos específicos Não é viável examinar as memórias de levantamento para estabelecer o escopo sem que haja o desenvolvimento contínuo de registros documentais. Eles, pelo menos, devem relacionar as conclusões a partir da análise dos requisitos em uma lista de requisitos funcionais.

Os requisitos não funcionais devem também ser considerados ao organizar as informações no nível de tarefa quando se especifica um escopo, pois é possível haver diferentes requisitos não funcionais sobre um mesmo tema se o compararmos no plano geral e no plano de uma tarefa ou grupo de tarefas em especial.

Por exemplo, é possível que o tempo médio aceitável para uma transação interativa para o software em geral seja de até dez segundos. Para as transações de consulta ao catálogo de produtos, há uma exigência maior e o tempo de resposta médio não deve ultrapassar dois segundos.

Portanto, para o objetivo de qualificar o escopo, no mínimo deve haver uma especificação documentando o estoque de requisitos funcionais e os requisitos não funcionais.

8.3.1.3.2. Descrever o comportamento dos elementos constituintes do escopo Consolidar o escopo como resumido até este ponto é um objetivo intermediário. O outro objetivo de informação no desenvolvimento dos requisitos é entender, revelar e conceber o passo a passo de cada um dos requisitos no escopo e as regras de negócio que se aplicam; enfim, aprofundar o escopo.

Quando se tem esse objetivo, e dependendo da estratégia utilizada, é possível alcançá-lo sem a necessidade de especificações detalhadas. Técnicas como a prototipação podem se revelar adequadas e com isso exigir um trabalho menor examinando documentos. Determinar quando isso ocorre deve considerar as condicionantes apresentadas no Capítulo 2, ao expor o nível de detalhe dos requisitos.

8.3.2. Decomposição da informação

Por volta do término da década de 1970 até o final da década de 1990, a principal estratégia de desenvolvimento de software era baseada na

decomposição funcional. Quando se estabelece que a análise de requisitos é também um processo de decomposição, não se deve interpretar dessa forma.

Deve-se interpretar que as partes interessadas expressam os seus requisitos durante a elicitación em diferentes níveis. Alguns deles são expressos como macroprocessos que agregam vários objetivos do usuário. São os requisitos que correspondem aos objetivos agregadores e eles podem indicar que: ➤

- o software deve facilitar ou responder integralmente pela consecução de todos os objetivos agregados; ➤

- o software deve fazer isso apenas para alguns deles, o que indica uma questão implícita porque não estabelece quais sejam eles; ou ➤

- não se sabe nem mesmo quais sejam esses objetivos mais específicos, o que também indica uma questão implícita.

Como exemplo, o requisito para “Controlar entrada e saída de pessoas e veículos nas instalações portuárias” citado anteriormente.

A análise de requisitos deve identificar essas possibilidades como questões a serem respondidas, dúvidas. Uma vez elucidadas, o requisito deve ser decomposto em seus elementos constituintes e revelados quais desses elementos são requisitos para o software em desenvolvimento.

8.3.3. Síntese da informação

As informações levantadas ao longo da elicitación de requisitos estão também na forma de fragmentos compartilhados entre as partes interessadas e os analistas. Um fragmento de requisito tipicamente carrega consigo questões ainda em aberto e que devem ser exploradas. Eles correspondem aos requisitos no nível de objetivo de subfunção.

Por exemplo, o requisito “Apenas alunos que tenham pelo menos 75% de presença devem receber o certificado de participação”.

Esse fragmento esconde algumas questões como: quais seriam os requisitos funcionais em que isso se aplica? Poderíamos pensar: no requisito funcional que descreve (ou indica) a emissão do certificado! Contudo, não se pode afirmar com segurança que seja o único. Dependendo do caso, é possível haver um requisito funcional com as suas particularidades, em que o

próprio aluno emite seu certificado. Também é possível haver outro requisito funcional – parecido, mas diferente – voltado para a equipe de gestão de cursos que permita solicitar o certificado de participação de qualquer aluno, de qualquer curso, em qualquer época.

Durante a análise de requisitos, esses fragmentos devem ser avaliados como regras de negócio que devem ser observadas pelo software em desenvolvimento. Diferentes partes interessadas podem ter expressado uma mesma regra de negócio de maneiras diferentes. A informação pode ser complementar entre si ou apresentar contradição quando descreve uma mesma regra de negócio. Dessa análise são elaboradas questões para elicitación ou sintetizadas especificações descrevendo as regras de negócio já resolvidas.

As regras de negócio organizadas na análise de requisitos devem ser alocadas aos requisitos funcionais em que se aplicam. Muitas vezes, as partes interessadas “jogam soltas” as regras de negócio e o analista deve trabalhar para que sejam descobertos esses requisitos funcionais em que elas se aplicam por meio da elicitación de requisitos.

Esses fragmentos também podem indicar rituais compartilhados como parte de outros requisitos funcionais. Por exemplo, várias partes interessadas indicam haver um ritual comum em diferentes transações bancárias para um terminal de autoatendimento. O seu propósito é identificar de maneira positiva que o usuário seja realmente quem diz ser e, com isso, prevenir fraudes. Os responsáveis pelas diferentes transações sabem apenas informar que esse ritual deve ser realizado; alguns o descrevem de um jeito, outros de maneira diferente.

A análise revela a necessidade de descobrir quem é a autoridade sobre esse ritual comum, como ele funciona e quais devem ser as transações que o usam. Uma vez de posse dessas respostas, o analista deve “sintetizar” essa informação em uma especificação funcional própria ou como parte de outra especificação.

Os resultados da análise não se limitam à síntese dos requisitos a partir desses dois tipos de fragmentos. Durante a análise, quando se juntam as peças para identificar ou elaborar um requisito da solução ou de transição, é

bem possível descobrir que estão faltando algumas peças na caixa do quebra-cabeças! Os modelos são ótimas ferramentas para esse fim.

8.4. Modelar e usar modelos para refinar a informação

Este tópico apresenta a elaboração de modelos como um instrumento para refinar a informação e revelar os requisitos que resolvem a necessidade de negócio.

8.4.1. O que é modelagem?

Requisitos isolados não costumam ser complexos; são o relacionamento e a interdependência entre esses requisitos isolados que levam à complexidade. Um modelo é uma representação visual e descritiva para levar informação a uma audiência; em especial, com o objetivo de suportar a sua análise, entendimento e comunicação. O modelo busca representar informação de maneira estruturada para organizá-la e transmiti-la de forma mais amigável.

8.4.2. Por que modelagem?

Um conjunto de requisitos inclui várias perspectivas pelas quais pode ser analisado. Os modelos ajudam a visualizar e resumir essa complexidade em requisitos priorizando uma perspectiva. Concentram a informação sobre uma determinada perspectiva, uma faceta da realidade, abstraindo-se das demais. Eles fornecem contexto para transmitir informação via diferentes visões dos requisitos. Representar (ou entender) a realidade, portanto, exige diferentes tipos de modelos.

Elaborar ou utilizar modelos preexistentes durante a análise permite refinar a informação. Isso porque os modelos também permitem confirmar o conhecimento desenvolvido, identificar lacunas de informação e eliminar informação redundante ou contraditória mais facilmente.

8.4.3. Como realizar a modelagem?

Os modelos podem tanto ser insumos quanto produtos na análise de requisitos. Portanto, há potencial de uso de modelos já elaborados e disponíveis desde antes do projeto, assim como a produção de modelos como consequência da atividade de análise.

8.4.3.1. Modelos preexistentes

O analista deve determinar quais modelos existentes podem ser usados como ponto de partida e quais ele deve desenvolver para alcançar os objetivos da análise.

Por exemplo, se há um modelo de processo descrevendo a colaboração entre diferentes atividades em direção a um objetivo de negócio, então alocar os requisitos (que já devem estar desenvolvidos no nível de tarefa, como explicado anteriormente) como parte das atividades facilita concluir que determinada atividade seja: ➤

- manual, porque não há e não deve haver requisito alocado a ela; ➤
- informatizada ou automatizada, porém não há requisito alocado a ela porque isso é responsabilidade de outro projeto; ➤
- mais bem investigada, porque não há requisitos para ela, mas percebe-se que sem isso as necessidades de negócio não são plenamente atendidas.

8.4.3.2. Modelos a elaborar

Ainda que não haja modelos preexistentes a se estudar, uma das responsabilidades da análise de requisitos é decidir quais técnicas de modelagem são mais adequadas e quando deveriam ser usadas.

Nos capítulos anteriores, algumas técnicas de modelagem foram apresentadas, como o diagrama de contexto, o glossário e a declaração de problema. Neste, outras serão apresentadas, indicando seus pontos fortes e as circunstâncias em que melhor se aplicam.

As técnicas exploradas neste capítulo são:



Especificação via sentenças textuais.



Histórias do usuário.



Decomposição funcional.



Modelo de processo.



Modelo de domínio.



Caso de uso.

8.4.3.3. Tipos de modelo quanto à forma

Há diferentes tipos de modelos. Um ou mais formatos podem ser escolhidos conforme o tipo de informação a ser modelada e o objetivo da modelagem. Quanto à forma, eles podem ser classificados como: ➤

Matrizes: uma matriz é usada para modelar um requisito ou conjunto de requisitos que tem uma estrutura complexa, mas uniforme. Matrizes também são usadas para priorizar requisitos e registrar outros atributos.



Diagramas: um diagrama é uma representação visual, em geral composto de figuras, de um requisito ou conjunto de requisitos. Um diagrama é especialmente útil para descrever a complexidade mais facilmente por imagens do que por palavras. Os diagramas podem também ser usados para delimitar fronteiras, para criar hierarquias e

classificar itens em categorias, e para apresentar a estrutura dos dados e seus relacionamentos.

8.4.3.4. Tipos de modelo quanto à informação

Os modelos têm o papel de prover informação em diferentes perspectivas, abstraindo-se dos detalhes das demais com o objetivo de com isso diminuir a sua complexidade inerente. A Figura 8.5 ilustra essa função dos modelos.

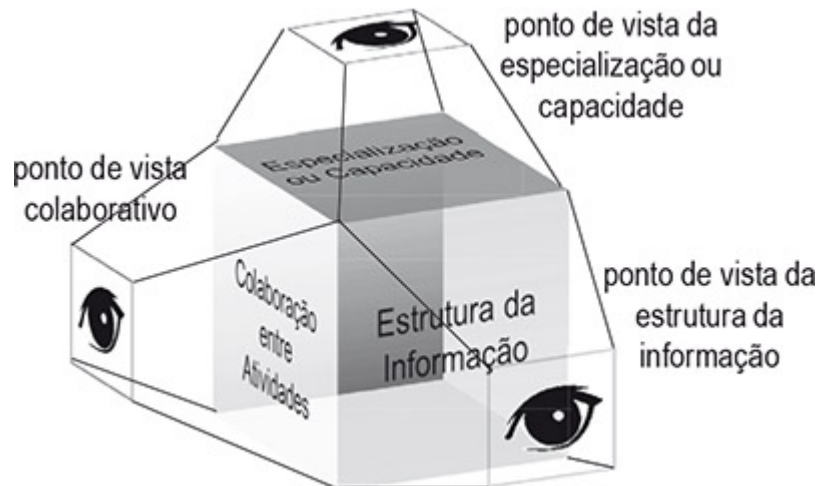


Figura 8.5. As diferentes visões representadas em modelos durante a análise.

Nesse sentido, do tipo de informação e do objetivo da modelagem, os modelos podem descrever os requisitos na perspectiva de: ➤

Colaboração: o assunto é a interação entre os requisitos, já associados aos objetivos de indivíduos realizando uma tarefa, posicionados em um plano no qual processos em diferentes níveis colaboram entre si na busca por um objetivo maior. São modelos que colocam em perspectiva a informação sobre o fluxo de atividades; modelos que representam uma sequência de ações, eventos ou caminhos que podem ser seguidos. Exemplos: modelo de processos, casos de uso e cenários, histórias do usuário, diagrama de sequência e diagrama de colaboração.

□

Especialização (ou capacidade): o assunto é a organização da informação com base em características e funções de uma

organização ou solução; são as afinidades entre diferentes responsabilidades que compartilham a necessidade por habilidades e conhecimentos comuns no desempenho de funções. Modelos deste tipo agrupam e dão estrutura aos requisitos conforme o seu papel no desempenho dessas funções. Exemplo: decomposição funcional e prototipação. Modelos com informações que representam organizações, grupos de pessoas, papéis e seus relacionamentos dentro de uma organização maior e para com a solução são especializações desse assunto. Por exemplo, a modelagem organizacional em um organograma; matrizes com papéis e permissões; e relação de partes interessadas.



Estrutura da informação: o assunto é a organização da informação em grupos coesos descrevendo conceitos sobre os quais há requisitos de armazenamento ou recuperação. Exemplo: dicionário de dados, glossário, modelo de domínio, diagrama de estados.

Ao exercitar a criação dessas diferentes perspectivas sobre os requisitos, o analista perceberá com mais facilidade pontos falhos da sua especificação.

8.4.3.5. Seleção de modelos

É comum haver múltiplas escolhas válidas e é bastante improvável que todos os modelos citados sejam usados em um mesmo projeto. Certamente mais de um tipo será usado na maior parte dos projetos. A escolha deve privilegiar a identificação de oportunidades para aperfeiçoar as operações do negócio. Alguns exemplos comuns de oportunidades que o analista possivelmente identificará incluem: ➤

Automatizar ou simplificar o trabalho realizado: tarefas relativamente simples, nas quais decisões são tomadas com base em regras estritas e inflexíveis, são candidatas principais a automação.



Melhorar o acesso à informação: fornecer maior quantidade de informação para a equipe que interage diretamente ou indiretamente com clientes, reduzindo assim a necessidade de especialistas. Os que tomam decisões podem não exigir esse nível de detalhe, mas deveriam estar conscientes de onde e a partir de quem eles podem obtê-lo se exigido. Normalmente, os que tomam decisões necessitam que lhes sejam fornecidos dados adquiridos e usados por pessoal operacional com a sua relevância e significado.



Reduzir a complexidade das interfaces: as interfaces são necessárias sempre que o trabalho é transferido entre sistemas ou pessoas. Por meio da redução de sua complexidade, pode-se aperfeiçoar o entendimento.



Aumentar a consistência do comportamento observável: diferentes profissionais podem manejar casos similares de maneiras bem diferentes, promovendo a insatisfação e frustração de clientes.



Eliminar redundância de informação: grupos de diferentes partes interessadas podem compartilhar necessidades em comum que podem ser atendidas com uma única solução, reduzindo assim o custo de implementação.

Como um exemplo de critério de seleção de modelos, um dos autores definiu uma política de qualidade na qual todo conceito de negócio armazenado pelo software que passasse por três ou mais estados conforme se transforma ao longo do processo de negócio deveria ter um diagrama de estados elaborado e limitado ao domínio do problema abordado. Cada transição de estados deveria, então, ser associada a um ou mais requisitos funcionais; caso não estivesse, deveria haver a confirmação junto à parte

interessada com autoridade para tal de que a transição não é um requisito no escopo do software em desenvolvimento.

A Figura 8.6 ilustra a implementação da estratégia associada à política de qualidade descrita em um cenário que tem como requisito de armazenamento os dados sobre o conceito Pessoa. Há quatro transições sem qualquer requisito associado. O que isso quer dizer?

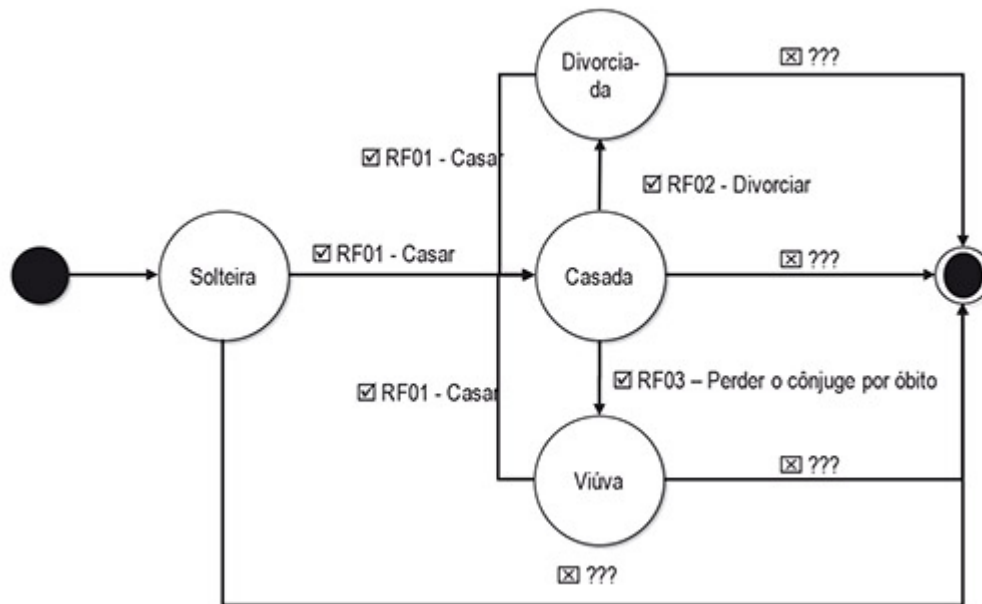


Figura 8.6. Exemplo de um diagrama de estados ilustrando as transições e os requisitos para o software que os abordam.

Provocar esse questionamento é ser proativo. A combinação de dois modelos – especificação em sentenças textuais e diagrama de estados, como no exemplo – permite descobrir obstáculos ao atendimento das necessidades de negócio com o investimento mínimo; em sua ausência, níveis de retrabalho nas disciplinas seguintes acumulam-se exponencialmente.

Observe que o modelo não deve ser um fim em si mesmo. Na empresa em que essa política foi implementada, anteriormente se exigia como um “entregável” o diagrama de estados. Centenas de folhas de papel, horas de esforço para elaborá-las, tramitar, validar e arquivar sem qualquer propósito para os vários elementos cujos únicos estados eram: ativo e inativo.

Fatores humanos, restrições organizacionais e a metodologia são fatores que devem ser considerados ao selecionar qual técnica de modelagem utilizar.

Eles são explorados a seguir.

8.4.3.6. Seleção de modelos e as restrições organizacionais

Um exemplo de restrição organizacional é não haver pessoal com habilidade em determinada técnica de modelagem e não haver recursos para adquirir essa capacidade para um projeto.

O uso de modelos gráficos tem maior eficácia quando associado a ferramentas de software, porém há muitos casos onde uma simples folha em branco cumpre o papel de elaborar um esboço. Contudo, tais ferramentas são fundamentais para a atualização de novas versões do modelo, ao passo em que se recebe retroalimentação a partir de versões intermediárias elaboradas.

Outro exemplo de restrição organizacional são os padrões sobre quais ferramentas de software utilizar na modelagem. Hoje esse é o principal fator a considerar, pois há muita variedade e disponibilidade de produtos para modelagem que vão do *freeware* a licenças de uso corporativo. Mesmo que haja aplicabilidade de dois determinados modelos em um cenário, é possível que restrições de tempo ou orçamento exijam que se priorize um em detrimento do outro. Essa priorização deve considerar outros fatores na seleção dos modelos utilizados: ➤

Timing: momentos iniciais privilegiam modelos que descrevem as necessidades de negócio, o domínio do problema e o escopo em nível geral; momentos intermediários na Engenharia de Requisitos privilegiam a definição do escopo de maneira específica; e momentos finais privilegiam o detalhamento dos itens no escopo. Por exemplo, um diagrama de casos de uso pode ser usado em momentos preliminares e com o objetivo de delimitar o escopo da solução em desenvolvimento. Da mesma forma, um diagrama de contexto cumpre o mesmo propósito. Considerando o nível de informação disponível nesse ponto do desenvolvimento, o diagrama de contexto representa a informação sem a necessidade de representar macroprocessos como se fossem casos de uso – quando não são.



Nível de abstração: conforme o nível de abstração dos requisitos – se eles são mais gerais ou mais específicos –, devem ser privilegiados modelos que acomodem o respectivo nível de abstração. Por exemplo, deve-se priorizar o uso de especificações em sentenças textuais enquanto os requisitos se desenvolvem ao longo da dialética de elicitação e análise quando há várias partes interessadas interagindo entre si e a equipe de desenvolvimento, ainda que seja possível elaborar histórias do usuário com o mesmo propósito.



Cobertura: não concentrar energias em modelos que enfocam uma determinada perspectiva em detrimento de outra que deixaria de ser abordada. Por exemplo, por que investir na elaboração de um número de modelos descrevendo uma perspectiva colaborativa e uma perspectiva da estrutura da informação quando não há modelo algum explorando a dimensão da especialização?

8.4.3.7. Seleção de modelos e fatores humanos

Um exemplo de fator humano é a maneira pela qual as pessoas adquirem informação sobre um assunto. Há pessoas cujo entendimento de um tema exige apoio visual, outras recebem melhor a informação textual. De maneira similar, há pessoas com maior facilidade de compreender conceitos abstratos, enquanto outras conseguem apenas entender conceitos abstratos quando materializados em objetos concretos.

A Tabela 8.1 ilustra o esquema de classificação de estilos cognitivos. Há um questionário desenvolvido para avaliar onde uma pessoa se enquadra nesse esquema de classificação denominado “Questionário Honey-Alonso de Estilos de Aprendizagem” (ALONSO, 2002).

Tabela 8.1. O esquema de classificação de estilos cognitivos.

Teórico	Reflexivo
• Aprendem melhor quando as coisas são apresentadas como parte de um sistema,	• Aprendem melhor com novas experiências, mas sem estar

modelo, teoria ou conceito. • Gostam de analisar e sintetizar. • Se algo é lógico, então é bom.	diretamente envolvidos nela. • Reúnem dados e os analisam com determinação para emitir conclusões. • Observam a atuação dos demais, escutam, mas não intervêm até tomarem pé da situação.
Pragmático • Ponto forte é a aplicação prática das ideias. • Descobrem o aspecto positivo das novas ideias e aproveitam a primeira oportunidade para experimentá-las. • Tendem a ser impacientes quanto há pessoas que teorizam.	Ativo • Envolvem-se totalmente e sem preconceitos em novas experiências. • Aproveitam o momento presente e se deixam levar pelos acontecimentos. • Tendem a se entusiasmar com o novo e a agir primeiro e pensar depois nas consequências.

8.4.3.8. Metodologia

Por fim, as diferentes metodologias de desenvolvimento de sistemas têm objetivos comuns: ➤

Retenção do conhecimento: o esforço de desenvolvimento de software produz itens como programas de computador, roteiros de configuração, bancos de dados *etc.* necessários à sua execução em um ambiente operacional físico. Produtos intermediários resultantes do trabalho intelectual se perderiam se não fossem registrados. Então, o papel da metodologia no âmbito da gestão do conhecimento é no mínimo reter esse trabalho para que este não se perca.



Melhoria contínua: o desenvolvimento de software é o resultado do encadeamento de várias atividades com objetivos específicos que não necessariamente devem seguir a mesma sequência e ter uma única forma de realização. A metodologia descreve práticas comprovadas e torna seu uso institucional, de forma que todos tenham uma orientação uniforme que capture os casos de sucesso à medida que a metodologia evolui.



Garantia de qualidade: a metodologia serve de modelo de referência para a verificação da conformidade na garantia de qualidade. A avaliação da qualidade de um produto depende de suas especificações. Ela também depende da qualidade do processo que orienta a sua produção. Essas avaliações, tanto do produto quanto do processo, necessitam de referências para o certo e para o errado. A metodologia cumpre este papel do processo, de maneira similar ao papel que as especificações que a metodologia define servem como referencial para avaliar a qualidade dos produtos.



Direcionamento do trabalho: a metodologia viabiliza a comunicação entre diferentes especialistas e organizações. O desenvolvimento de sistemas na atualidade é resultado de diversas especializações funcionais verticais, inclusive podendo envolver várias organizações. A metodologia é o que provê os meios para a integração horizontal entre os produtos de trabalho intermediários de cada especialização em direção ao software disponível, funcional, em um ambiente operacional físico.



Fator normalizador: a metodologia permite estabelecer, negociar e avaliar de maneira homogênea níveis de serviços de diferentes organizações internas e externas. Dessa forma, é possível planejar e monitorar o desempenho, ainda que haja uma variedade de organizações e especialidades interagindo. A metodologia cumpre o papel, nesse contexto, de uniformizar o trabalho.

Em função de um ou mais desses objetivos, a seleção de modelos sofre impactos que podem limitar o horizonte de escolhas.

8.5. Especificar para documentar os requisitos

A modelagem e a especificação são coisas diferentes, pois a primeira coloca ênfase no desenvolvimento (revelação) dos requisitos e na sua comunicação com as partes interessadas no negócio. A segunda coloca ênfase na transmissão da informação para a equipe de desenvolvimento; ainda que ambas as atividades exijam a produção de informação compreensível por todas as partes.

8.5.1. O que é especificação?

A especificação de requisitos documenta os diferentes tipos de requisitos. O formato da documentação resultante da especificação de requisitos depende da organização, das necessidades do projeto e do ciclo de vida utilizado.

Por exemplo, as histórias do usuário serão os documentos que descrevem o estoque de requisitos a ser entregue quando se utiliza uma abordagem ágil de desenvolvimento; por sua vez, se uma abordagem derivada do Processo Unificado for utilizada, então as listas de requisitos cumprem esse papel.

A especificação de requisitos não precisa necessariamente cobrir todo o escopo, mas pode representar a informação disponível em determinado momento no tempo.

8.5.2. Por que especificação?

O PMI (2015) fornece uma relação bastante completa para especificar requisitos. Ela está transcrita a seguir: ➤

- Estabelece uma linha de base para: ✓
 - validar as necessidades das partes interessadas; ✓
 - definir a solução para as necessidades de negócio; ✓
 - a evolução da solução.



Fornece insumo para:



a equipe de projeto (*design*), os desenvolvedores, analistas de testes e garantia de qualidade; ✓

o manual do usuário e outros tipos de documentação, sejam eles impressos ou embarcados.



Fornece detalhamento de suporte para: ✓

acordos contratuais, quando aplicáveis (por exemplo, os requisitos são insumos centrais para a declaração de trabalho ou para uma solicitação de proposta); ✓

auditoria em indústrias reguladas e projetos de alto risco que têm como exigência seus requisitos documentados.



Viabiliza reutilizar informação por outros times de projeto que necessitam de um entendimento dos requisitos enquanto o projeto está em processo de implementação ou posteriormente.

O PMI também registra, de maneira muito apropriada, que, apesar da importância de documentar os requisitos, deve-se manter em mente que esta é apenas uma de várias técnicas para garantir o consenso entre todas as partes interessadas quanto ao comportamento esperado da solução (como explorado anteriormente) e que a documentação não deveria substituir a comunicação e a colaboração.

8.5.3. Quando elaborar uma especificação?

Há vários momentos em que se faz necessário elaborar documentos com especificações de requisitos. Dependendo do momento em que se esteja, há diferentes necessidades de informação, cujas respostas devem ser capturadas em especificações de requisitos. O esquema apresentado no

Capítulo 6 descreve um processo genérico com três marcos associados e que é útil para discutir a especificação de requisitos conforme esses momentos.

Descrevem-se as especificações relativas aos objetivos de informação que devem ser alcançados até o primeiro marco: a qualificação do domínio do problema, as necessidades de negócio e um escopo em termos de áreas funcionais, organizações afetadas e macroprocessos afetados; possivelmente uma lista de características para a solução ainda muito geral e pouco específica.

Os próximos tópicos abordam ações para atingir os outros objetivos de: ➤

- qualificar especificamente quais são as tarefas transferidas para o software; e ➤

- descrever o comportamento que se espera do software no que tange a essas tarefas.

8.5.4. Como elaborar uma especificação?

Nesse sentido, o mínimo do mínimo que se necessita especificar são os requisitos identificados no nível do objetivo do usuário, expressos em sentenças textuais. Mesmo no desenvolvimento usando abordagens ágeis e que priorizam a colaboração, há necessidade de especificações associadas a esse mínimo. Os adeptos do Scrum, por exemplo, têm no *Product Backlog* a especificação de histórias do usuário que compõem o escopo inicial para o desenvolvimento.

Essas sentenças textuais devem:



- expressar um e apenas um requisito por vez; ➤

- evitar cláusulas condicionais complexas; ➤

- usar uma terminologia consistente; ➤

- expressar requisitos em uma sentença com verbo em voz ativa; ➤

- indicar claramente o que ou quem é responsável pela ação realizada pelo requisito.

Para requisitos associados à manutenção de dados cadastrais – os denominados CRUD (*Create, Read, Update, Delete*) –, a prática comum é usar o verbo “manter”. Por exemplo, “Manter Produto” (ou “Manter Cadastro de Produto”) é uma sentença que remete a incluir, consultar, atualizar e excluir um registro de produto.

As sentenças não devem assumir que o leitor tenha um conhecimento prévio do domínio.

Por exemplo, a sentença da Tabela 8.2 expressa vários requisitos e inclui cláusulas condicionais, além de não incluir um verbo que caracterize a ação principal realizada.

Tabela 8.2. Como não especificar requisitos por sentenças textuais.

Identificador: RF001	Requisito: Controle de Portaria
Para realizar o controle de entrada e saída na portaria deve ser realizado o cadastro do visitante através dos seguintes atributos: Nome, Registro Geral e Imagem. Caso a visita tenha sido liberada pelo condomínio, então podem ser registradas a data e a hora em que o visitante acessou a portaria. Ao sair do condomínio, devem ser registradas a data e a hora em que o visitante saiu, mas só deve ser permitido para visitante com registro de entrada e sem registro de saída.	

Os mesmos requisitos deveriam ser reestruturados conforme a Tabela 8.3.

Tabela 8.3. Resultado da reformulação das sentenças textuais.

Identificador: RF001	Requisito: Manter Cadastro de Visitante
O condômino cadastra o visitante registrando Nome e Registro Geral.	
Identificador: RF002	Requisito: Liberar Acesso de Visitante
O condômino cadastra a liberação de acesso selecionando um visitante já cadastrado e indica o período (data e hora de início e data e hora de fim) para realizar a visita.	
Identificador: RF003	Requisito: Registrar Entrada de Visitante
O porteiro ou guarda verifica se a documentação apresentada pelo visitante confere com os dados apresentados na liberação de acesso (Nome do Visitante e Registro Geral) e registra a data e a hora em que o visitante entrou no condomínio.	

A representação das condicionantes que se aplicam em um nível superior, no encadeamento dos processos em que os requisitos se inserem, deve ser feita por meio da modelagem de processos, por exemplo.

O detalhamento dos requisitos identificados nas sentenças textuais pode ser realizado por meio de casos de uso e especificações de regras de negócio ou mesmo outros métodos não documentais, como a prototipação. As especificações de mudança/manutenção seguem a mesma lógica.

Deve-se ressaltar que alcançar o marco de consolidação do escopo não implica em esgotar a identificação de todos os requisitos, assim como alcançar o marco do detalhamento dos requisitos não implica em esgotar a descrição do passo a passo e das regras de negócio que se apliquem a esses requisitos.

Ou seja, ainda que o marco de consolidação do escopo seja alcançado, é possível que ainda haja itens de importância secundária em cinza – escondendo vários requisitos funcionais e não funcionais a descobrir como dentro do escopo (preto) ou fora (branco) em requisitos como “emitir relatórios gerenciais”. Ainda que o projeto tenha ultrapassado o marco citado, novas especificações indicando quais requisitos funcionais farão parte do escopo serão elaboradas como histórias do usuário ou elementos em uma lista de requisitos mais refinada.

Embora isso não seja recomendável, há casos em que as especificações de requisitos são elaboradas após o projeto. Sua utilidade é reduzida. Porém, em uma perspectiva de evolução do produto, essas especificações cumprirão o papel de reter conhecimento produzido no projeto, facilitando as manutenções futuras.

8.6. Verificação de requisitos

Por mais cuidado que se tome ao elaborar uma especificação de requisitos, o seu próprio autor nunca será uma pessoa isenta para avaliar se o trabalho foi bem feito. Erros quase sempre estarão presentes (lembre-se de que não existe especificação perfeita), e quem elaborou o trabalho costuma ter mais dificuldade para perceber essas falhas. Por isso a avaliação da especificação de requisitos por uma terceira pessoa ajuda a filtrar problemas e a melhorar

a qualidade do trabalho de requisitos. Este é o papel das atividades de verificação e validação de requisitos.

8.6.1. O que é verificação?

Verificar requisitos é comparar os produtos da modelagem e da especificação de requisitos com modelos de referência e adotar ações para apontar as não conformidades para que estas possam ser justificadas ou corrigidas.

A diferença entre a verificação e a validação de requisitos pode ser resumida da seguinte forma: ➤

Verificação de requisitos: as especificações atendem ao padrão?
Suas atividades não exigem a participação do cliente.



Validação de requisitos: as especificações atendem ao cliente?
Suas atividades exigem a participação do cliente.

8.6.2. Quem realiza a verificação?

O próprio autor da especificação não deveria ser o responsável pela verificação do seu trabalho, pois sua análise quase sempre é enviesada. A verificação de requisitos é uma atividade essencialmente interna à equipe de projeto. Há organizações com profissionais especializados nessa atividade, eventualmente parte de um grupo denominado de Garantia de Qualidade de Software (*Software Quality Assurance* – SQA). O alcance das atividades é mais amplo porque são verificados também produtos de outras disciplinas, não só de requisitos.

8.6.3. Por que verificação?

A verificação de requisitos busca garantir que as especificações de requisitos e seus modelos atendam ao padrão de qualidade para que possam ser utilizados de forma eficaz nas atividades seguintes do projeto. Ela

também cumpre o papel de preparar os requisitos para a validação. Tentar validar os requisitos sem antes verificá-los pode implicar em desviar a atenção da validação para questões que a verificação poderia ter tratado antes.

8.6.4. Quando realizar a verificação?

A verificação tem início quando um conjunto de especificações é considerado feito e os responsáveis pela gerência de requisitos decidem juntá-lo em um pacote de requisitos para encaminhá-lo para desenvolvimento. Trata-se de uma avaliação final do pacote de requisitos, antes de proceder com a validação junto às partes interessadas.

8.6.5. Como realizar a verificação?

A verificação acontece por meio da inspeção dos resultados da modelagem e especificação usando modelos de referência internos ou externos; e pela comparação entre diferentes elementos da documentação associada.

O CMMI apresenta a verificação com objetivos específicos a serem alcançados. Destaca-se um subconjunto em uma visão de atividades para explorar como verificar requisitos. Essas atividades estão divididas em dois grupos, como a seguir: ➤

Preparar para verificação:

✓

Selecionar produtos de trabalho para verificação.

✓

Estabelecer critérios e procedimentos de verificação.

✓

Estabelecer ambiente de verificação.

□

Verificar produtos de trabalho selecionados: ✓
Executar a verificação.

✓

Analisar resultados da verificação.

Das duas primeiras atividades, recomenda-se criar no plano corporativo diretrizes aplicáveis a todos os tipos de projetos desenvolvidos. A cada projeto cabe reavaliar esses dois pontos para identificar oportunidades de adequação. A Figura 8.7 ilustra a organização dessas atividades como descrito.

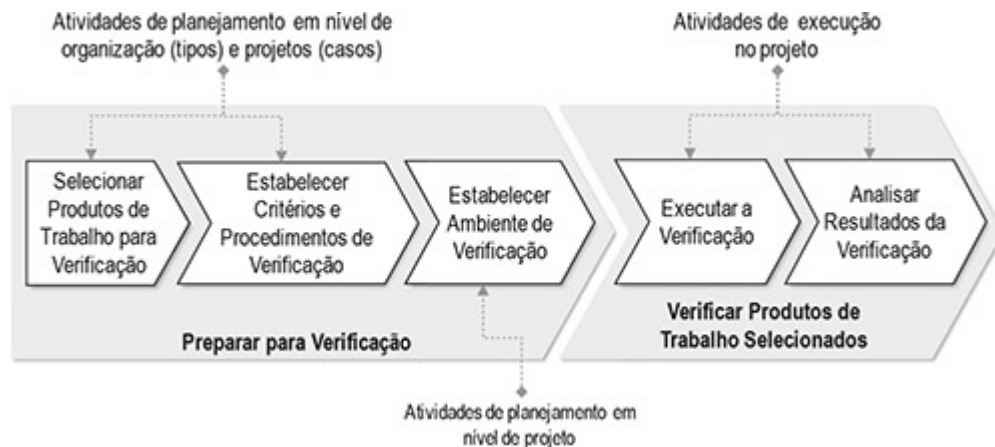


Figura 8.7. A dinâmica do trabalho de verificação de requisitos.

A atividade de **Selecionar Produtos de Trabalho para Verificação** permite a identificação de tipos de produtos de trabalho para verificação (quando executada no plano da organização) ou casos de produtos de trabalho (quando executada no plano de um projeto), métodos a serem usados para executar a verificação e as exigências que devem ser satisfeitas para cada produto de trabalho selecionado.

A atividade de **Estabelecer Ambiente de Verificação** permite a determinação do ambiente a ser usado para dar sequência à verificação. Seus produtos respondem à questão: o que é necessário para que seja realizada a verificação conforme os outros planos?

A atividade de **Estabelecer Critérios e Procedimentos de Verificação** permite o desenvolvimento de procedimentos e critérios que devem estar alinhados com os produtos de trabalho, métodos e características do ambiente de verificação. As atividades em **Verificar Produtos de Trabalho Selecionados** conduzem a verificação de acordo com os métodos, procedimentos e critérios definidos.

8.6.5.1. Verificação de um modelo em especial e sua integração com os demais

A seguir apresenta-se orientação prática para facilitar a verificação no nível de projeto ou corporativo. A intenção não é descrever um processo, mas apresentar os elementos que permitam a organização das atividades citadas. Deve-se verificar se cada modelo/especificação: ➤

Não apresenta elemento omissos em outros modelos. Compare cada modelo, parte do pacote em verificação, com os outros modelos disponíveis no projeto. Procure por elementos que são mencionados no modelo ora em verificação e que estão omissos em modelos que exploram outras perspectivas. Por exemplo, em um diagrama de atividades há etapa de emissão de certificados; contudo, não há requisito funcional ou caso de uso algum para este fim.



Usa uma terminologia para um mesmo conceito de negócio de maneira consistente em todos os modelos. O glossário, cujo uso se recomenda como obrigatório, é um excelente instrumento para esse fim. O modelo de domínio é outro recurso que cumpre o mesmo objetivo. O uso de uma terminologia inconsistente pode indicar um erro ou a necessidade de elicitação adicional para identificar requisitos, até então, omissos. Por exemplo, no requisito funcional “RF032 – Registrar Avaliação do Pedido” é utilizado o termo “Pedido”. Já no requisito “RF012 – Encaminhar Solicitação para Produção” utiliza-se “Solicitação”. Ambos os requisitos se referem ao mesmo assunto? Ou uma coisa é um pedido e outra coisa é uma solicitação? O glossário deveria dar essa resposta. Deve-se resolver

todas as discrepâncias, corrigindo a terminologia ou ajustando os modelos conforme necessário.



As necessidades de informação para o tipo de modelo em análise estão atendidas de maneira completa. Por exemplo, há uma especificação para Modelo e Notação de Processo de Negócio (*Business Process Model and Notation* – BPMN) adotada para descrever a dimensão colaborativa dos requisitos. Ou então, para o mesmo fim, existe o Diagrama de Atividades como definido na Linguagem Unificada de Modelagem (*Unified Modeling Language* – UML). Um deles é o modelo de referência na organização ou projeto. Então, verificar um modelo colaborativo é avaliar se ele observa o padrão e não omite informação definida como obrigatória por ele.



Há requisitos ausentes identificados pela comparação do modelo em análise com os resultados da elicitação (memórias de levantamento validadas). O item de verificação e a diretriz técnica associada, a seguir transcritos, ilustram critérios de verificação sobre a avaliação do escopo: *Item de Verificação 42. O Quadro de Requisitos Funcionais (Escopo) está COMPLETO? (02.DT.48).*

02.DT.48 – *Verificar se o Quadro de Requisitos Funcionais (Escopo) está completo.*

O Quadro de Requisitos Funcionais (Escopo) está CORRETO quando todos os elementos têm fundamentos para sua identificação nos documentos de memória do levantamento que devem permitir identificar as tarefas e os serviços do usuário transferidas para o software a partir de manifestações espontâneas dos envolvidos, ou como resultado da aplicação de técnicas de elicitação por parte da equipe de desenvolvimento. Está COMPLETO quando o mesmo se verifica em sentido inverso.



Todos os eventos externos para os quais o software deve prover uma resposta a partir de seu processamento ou resultados necessários que devem ser produzidos pelo software (independentemente de algum evento externo) devem ser identificados e descritos em todas as suas variações conforme o nível de detalhe definido para o projeto. A Figura 8.8 ilustra eventos para os quais o software deve prover resposta.

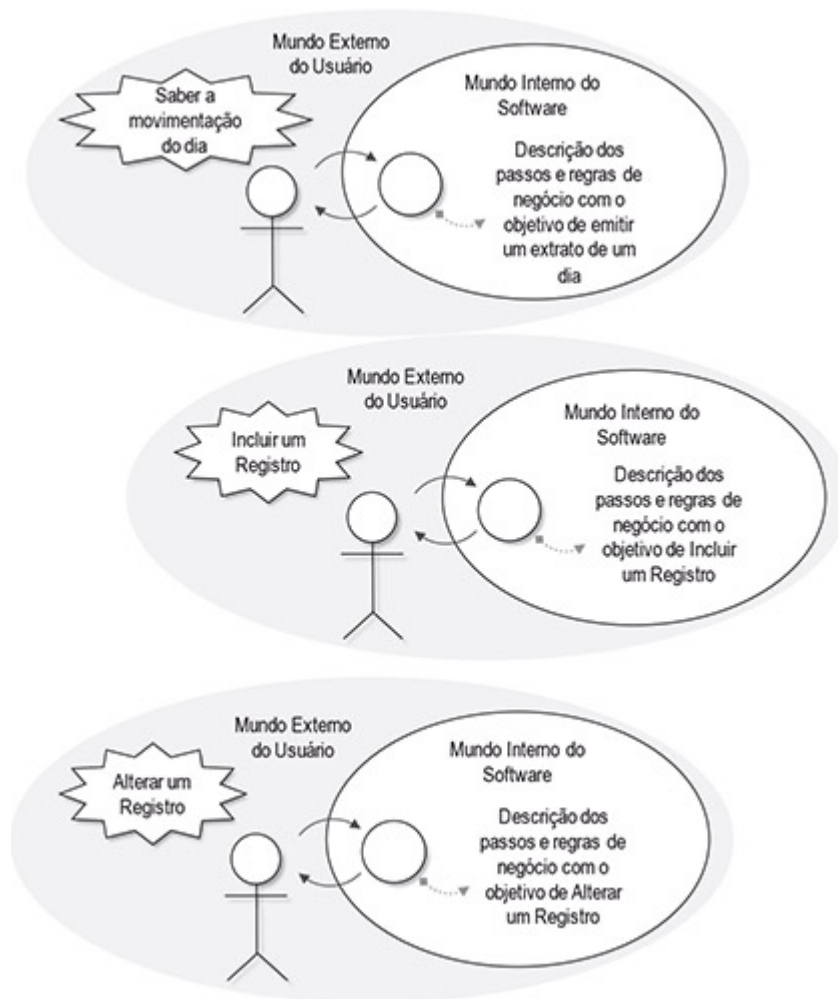


Figura 8.8. Exemplos de eventos externos e independentes para os quais o software deve prover respostas.

Na literatura sobre verificação de software, percebe-se uma ênfase na “sintaxe” e é comum encontrar exemplos como: *O nome do caso de uso inicia com um verbo no infinitivo, começando com letra maiúscula?*

Havendo mais de uma palavra, estas também começam com letra maiúscula (exceto preposições)?

Observar uma sintaxe é relevante, mas vive-se uma época em que fazer rápido é mais importante do que fazer “certo” quando o “errado” for violar regras como as citadas. Ainda mais se for o caso de colocar uma preposição iniciando com letra maiúscula!

8.6.5.2. Verificação da definição do escopo na lista de requisitos funcionais

Quando o objetivo é verificar se a especificação de requisitos cumpre o propósito de definir o escopo de forma ampla e inequívoca, certifique-se de que cada requisito funcional é: ➤

identificado no nível de objetivo do usuário. O requisito no nível agregador que não permita a identificação de todas as tarefas contidas deve ser rejeitado. O requisito no nível de subfunção que não cumpre o propósito de tratar expectativas críticas das partes interessadas também deve ser rejeitado. Ainda que se permitam essas exceções, isso deve se manter dentro de uma regra 80/20 (no máximo 20% dos requisitos nos níveis agregador e subfunção); ➤
denominado a partir de um verbo em voz ativa em uma frase que representa o objetivo do usuário. Nomes genéricos não estabelecem as expectativas do leitor nem proveem um ponto focal ao qual as pessoas possam referenciar de forma conveniente.

8.6.5.3. Verificação da descrição do requisito funcional – o detalhamento do escopo

A verificação associada ao detalhamento do escopo deve sempre estar associada ao nível de detalhe definido para as especificações no projeto ou produto. Certifique-se de que cada requisito funcional seja descrito de maneira que: ➤

haja um cenário com a história de sucesso em direção ao objetivo do usuário sem a consideração de possíveis falhas e, adiante, haja fragmentos de histórias que apresentem quais condições alternativas poderiam acontecer; ➤

sejam capturadas todas as falhas e alternativas que devem ser manipuladas. Um requisito funcional pode ter vários fluxos alternativos. Desconsiderar alguns desses fluxos significa que o desenvolvedor não entenderá apropriadamente o comportamento exigido para o software e aumentam as chances de que o software seja entregue deficiente. Preste atenção em especial em lógicas de desdobramento comum – nenhum registro encontrado, um e apenas um encontrado ou mais de um encontrado. Por exemplo, o requisito funcional “RF032 – Registrar Avaliação do Pedido” corresponde ao processamento esperado pelo software em resposta a um evento externo – a decisão sobre a avaliação que um faturista fez do pedido. Esta única decisão pode envolver diferentes caminhos, como ilustrado na Figura 8.9;

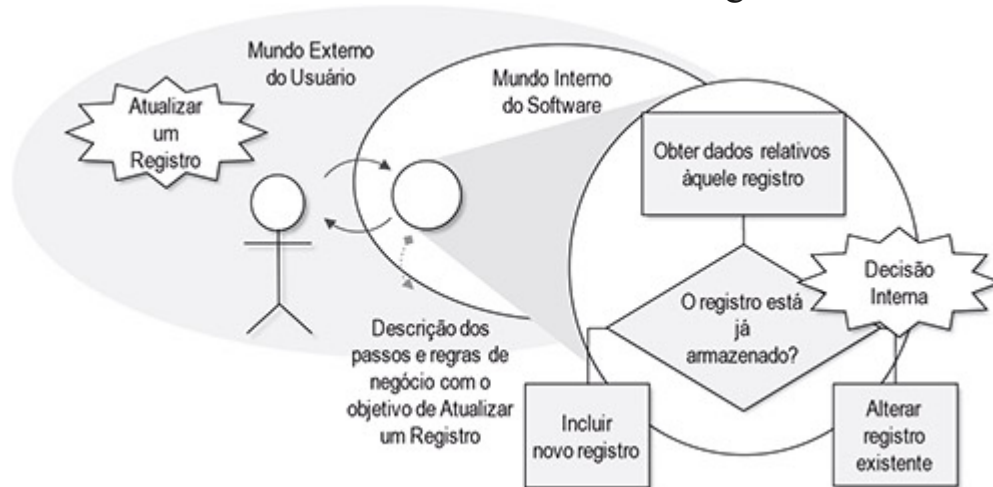


Figura 8.9. Diferentes caminhos em um mesmo requisito funcional.



não haja cenários que incluam em seu corpo requisitos não funcionais. Eles podem facilmente promover a desordem e prejudicar a clareza da informação. É necessário haver anexos com informação sobre os requisitos não funcionais que se aplicam; ➤ o leitor seja capaz de seguir facilmente o caminho por um cenário em particular no qual está interessado – caso contrário, ele possivelmente ficará frustrado ou perderá informação importante; ➤

a terminologia usada ao expressar o requisito seja compreensível para todas as partes interessadas e consistentes com os termos usados dentro da organização. Especificações de requisitos que são complicadas demais para leitores não técnicos, ou imprecisas para os desenvolvedores, são deficientes e potencializam sistemas pobremente construídos, inadequados. Os requisitos devem ser escritos de forma legível o suficiente para que as partes interessadas se deem ao trabalho de ler e avaliar, e preciso o suficiente para que os desenvolvedores entendam o que estão construindo. Utilize exemplos onde for apropriado para clarificar e fortalecer a compreensão; ➤

não haja duas ou mais condições descritas em separado que tenham o mesmo efeito quando se aplicam; ➤

não haja passos excessivamente grandes ou excessivamente pequenos, ou que obscureçam seu objetivo e sejam difíceis de ler e compreender. A descrição de cada cenário deveria possuir de três a nove passos. Idealmente, eles estão todos em níveis similares, um nível de abstração abaixo do nível de objetivo do usuário; ➤

cada passo indique claramente qual ator (em geral, um usuário e o próprio software) está executando uma ação e o que o ator deve conseguir. Ambos os leitores e desenvolvedores se confundem sobre o comportamento do sistema se esses dois pontos não estiverem claros; ➤

cada cenário conte uma história direta e simples, sem desdobramentos. Elimine de um cenário passos que não avançam o ator e simplifique passagens que distraiam o leitor da progressão da história; crie um fragmento descrevendo alternativas; ➤

não inclua detalhes de implementação, neutros em relação à tecnologia, pois fazer diferente aumenta a complexidade e obscurece o seu objetivo. Deve-se isolar o leitor de detalhes específicos de tecnologia, deixando-os livres para colocar foco no comportamento essencial do sistema. Isso não implica em dizer que não é necessário se preocupar com essa dimensão, apenas que deve ser descrita em separado.

8.6.5.4. Técnicas úteis à verificação

As técnicas citadas a seguir auxiliam direta ou indiretamente na verificação de requisitos. Optamos por explicá-las em partes específicas do livro, pois algumas têm utilidade em vários momentos da Engenharia de Requisitos. Portanto, aqui vamos apenas citá-las comentando no que podem ser úteis à verificação de requisitos. Vejamos: ➤

Controle de questões: se existem questões pendentes (ou em aberto) no controle de questões, significa que o trabalho de elicitação ainda não foi concluído para o projeto todo. E possivelmente podem indicar que há lacunas em requisitos em uma iteração específica. Consequentemente, o trabalho de análise também não se concluiu. E, logicamente, a especificação de requisitos não pode ser considerada completa.



Glossário: favorece o uso de um vocabulário consistente na especificação e familiar às partes interessadas.



Matriz de rastreabilidade: ajuda a verificar se existem requisitos de mais alto nível que ainda não foram tratados, além de detectar requisitos que estão sobrando, ou seja, que não deveriam fazer parte do projeto.



Checklists (listas de verificação): definem uma abordagem padrão para a identificação de problemas na especificação, apontando itens a serem seguidos pelo responsável pela verificação e pelo próprio autor da especificação.



Revisão (ou inspeção): usa uma visão externa ao autor da especificação para avaliar se a especificação consegue transmitir a mensagem pretendida por ele.



Geração de casos de teste: os casos de teste são importantes para a verificação do produto de software que será entregue, mas sua elaboração tem como efeito colateral benéfico a identificação de falhas na especificação, por exemplo, identificando requisitos não testáveis ou incompletos. Logo, a elaboração de casos de teste funciona como uma revisão da especificação.



Medição de pontos de função: o resultado da medição não tem muita utilidade para a verificação de requisitos, e sim para planejamento, acompanhamento e controle do projeto (VAZQUEZ, 2013). Porém, o processo de medição traz como efeito colateral benéfico a identificação de diversas falhas na especificação de requisitos. Isso porque o processo de medição nada mais é que também uma revisão estruturada da especificação dos requisitos (DEKKERS, 2001). A experiência dos autores mostra que essa abordagem é muito útil para detectar falhas na especificação de requisitos de: consistência, clareza e completude.

8.7. Validação de requisitos

A validação de requisitos é um trabalho de garantia de qualidade na Engenharia de Requisitos que busca assegurar que todos os requisitos especificados estejam alinhados com os requisitos de negócio. Ou seja, procurar garantir que todas as necessidades de negócio das partes interessadas no escopo do projeto serão satisfeitas. A finalidade é garantir que a especificação defina o produto certo a ser desenvolvido pelo projeto. Em resumo, avalia se o software satisfaz o cliente. Contextualizando a validação de requisitos nos processos de gerenciamento de projetos descrito no *PMBOK® Guide*, ela seria uma atividade componente do processo Validar o Escopo.

As técnicas citadas a seguir auxiliam direta ou indiretamente na validação de requisitos. Optamos por explicá-las em partes específicas do livro, pois algumas têm utilidade em vários momentos da Engenharia de Requisitos. Portanto, aqui vamos apenas citá-las e comentar em que podem ser úteis à validação de requisitos. Vejamos: ➤

Protótipos: talvez seja a maneira mais eficaz de validar requisitos. Ao apresentar uma proposta concreta do que será entregue como produto, a parte interessada consegue avaliar com mais interesse e atenção a solução proposta. É onde provavelmente se obtém um *feedback* mais rico do quão correta é a solução proposta.



Checklists (listas de verificação): definem uma abordagem padrão para a identificação de problemas da especificação, apontando itens de verificação a serem seguidos pelo responsável pela verificação. Embora seja uma técnica mais diretamente relacionada à verificação de requisitos, pode também ser útil para orientar o trabalho de validação pelo cliente. O que muda na abordagem de *checklist* de verificação para o de validação é que provavelmente as listas de itens a verificar são distintas nos dois casos. Além do fato de que, na verificação, quem executa a lista é a equipe do projeto e na validação são as próprias partes interessadas chave.



Inspeção (ou revisão): usa uma visão externa ao autor da especificação para avaliar se a especificação consegue transmitir a mensagem pretendida por ele. É uma técnica que se aplica tanto à verificação quanto à validação. A diferença é que na verificação o revisor é da equipe do projeto e na validação o revisor é uma parte interessada chave.

8.8. Técnica: histórias do usuário

Uma história do usuário é uma breve declaração que descreve algo que o sistema deve fazer para o usuário. É um tipo de especificação de requisitos adotado por muitas equipes de projetos “ágeis”. Como, por exemplo, na história do usuário representada na Figura 8.10.

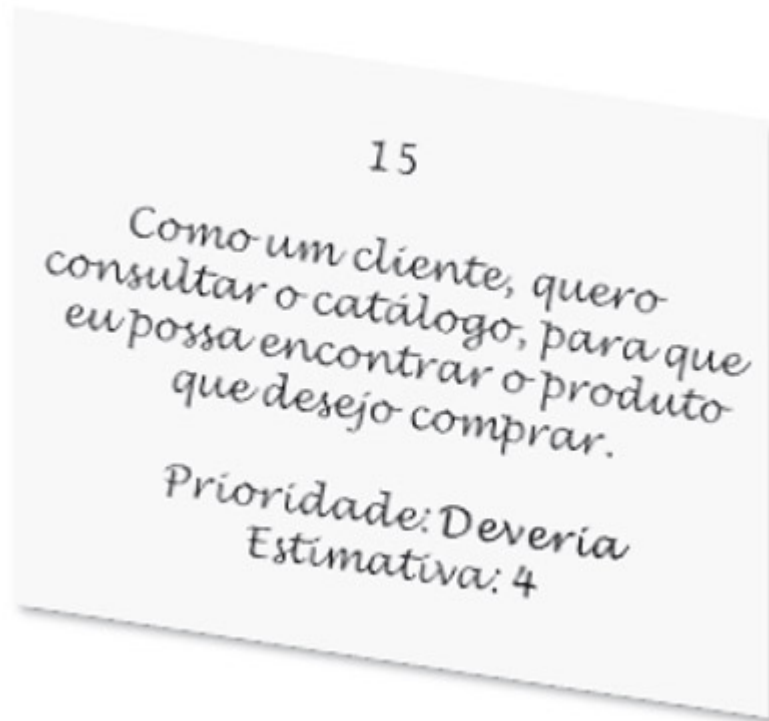


Figura 8.10. Exemplo de história de usuário: “como um cliente, quero consultar o catálogo, para que eu possa encontrar o produto que desejo comprar”.

Considerando a estrutura de tipos para classificação de requisitos apresentada, a história do usuário se posiciona como parte dos requisitos da solução, especificamente um requisito funcional, inicialmente no nível de objetivo do usuário. Coloca-se a história do usuário apenas inicialmente nesse nível, pois ela pode eventualmente ser subdividida como uma decisão de projeto para que sua implementação caiba em uma iteração.

Ela se restringe a definir o escopo sem entrar no detalhamento do passo a passo ou das regras de negócio que se aplicam à tarefa do software. Os detalhes do comportamento do sistema são desenvolvidos por meio de interações entre a equipe de desenvolvimento e o dono do produto; pela definição de um critério de aceitação. Isso não muda o seu papel como uma representação dos requisitos da solução. Contudo, o processo pelo qual a história do usuário é identificada, elaborada e mantida difere do processo

tipicamente associado a outras especificações que ocupam espaço similar, como listas de requisitos ou diagramas de casos de uso.

O uso de histórias do usuário foi introduzido como uma unidade de funcionalidade da *Extreme Programming* (XP). O progresso é demonstrado pela entrega de código testado e integrado que implementa uma história do usuário. Uma história do usuário deveria ser compreensível e de valor na perspectiva dos clientes, passível de testes pelos desenvolvedores e pequena o suficiente para que os programadores possam construí-la em uma iteração (tipicamente, entre duas e quatro semanas em projetos ágeis).

Hoje o método é parte integrante de outras abordagens, como o Scrum, na arena da gerência de projetos. Enquanto no XP promove-se a elaboração pelo próprio usuário, no Scrum quem as elabora é o dono do produto (*Product Owner*) a partir de informações elicitadas junto a outras partes interessadas.

8.8.1. Por que histórias do usuário?

Ao especificar requisitos por meio de histórias do usuário, cria-se um ambiente de propriedade das partes interessadas, que estabelecem prioridades e alocam recursos de forma incremental e interativa.

Porque a história do usuário não é uma documentação detalhada (e não é este seu objetivo), seu uso força a colaboração entre os membros da equipe para que a história possa ser compreendida por todos. Pelo mesmo motivo, exige pouco esforço em sua manutenção e exige que o valor entregue seja claramente articulado. Apesar desses benefícios, ela pode não ser adequada para ambientes mais burocráticos ou quando o nível de documentação do projeto é imposto.

O propósito da simples retenção do conhecimento é um dos fatores invocados como justificativa nessa imposição. Esse tipo de coisa apenas se justificaria se houvesse a intenção de investir tempo e recursos na manutenção de documentação detalhada atualizada. Caso contrário, ela ficará obsoleta. Ou seja, se não houver intenção de investir tempo e recursos nessa atualização, desperdiçam-se recursos em um detalhamento que basicamente cumpre o papel de um arquivo morto.

Um cuidado que deve estar presente quando se trabalha com histórias do usuário é a observação de requisitos não funcionais. A história do usuário não aborda explicitamente como documentar requisitos não funcionais.

8.8.2. Como elaborar uma história do usuário?

O estilo para redação de uma história do usuário é livre. Não há um modelo de referência que estipule um formato padronizado para sua elaboração. Contudo, a história do usuário deve responder a três perguntas: ➤

Quem se beneficia? Resposta indica a parte interessada que se beneficia da história do usuário (ator).



O que se quer? Resposta deve prover uma visão alto nível da funcionalidade para o usuário (descrição).



Qual é o benefício? Resposta indica o valor de negócio que a história proporciona (por quê).

As histórias do usuário devem ser escritas no espaço equivalente a um cartão; isso porque dessa forma limita-se a sua extensão, obrigando a produção de um texto claro e objetivo. Muitas empresas utilizam modelos pré-concebidos e há autores que recomendam escrever histórias do usuário no formato: Como (*papel*), eu quero (*algo*) para (*me beneficiar*).

8.8.2.1. INVEST

Um critério de qualidade para uma história do usuário é definida pelo acrônimo INVEST, descrito na Tabela 8.4.

Tabela 8.4. Critérios de qualidade INVEST para elaboração de histórias do usuário.

Letra	Significado	Descrição
I	Independente (<i>Independent</i>)	A observação do nível de objetivo do usuário ao identificar uma história do usuário garante que ela seja independente. Não importa como a história do usuário se encadeia com outras em um nível mais

		alto. Isoladamente, ela pode ser desenvolvida, testada e até mesmo entregue.
N	Negociável (<i>Negotiable</i>)	Em abordagens com maior orientação ao planejamento, o papel de uma lista de requisitos ou diagrama de casos de uso é limitar o escopo da solução em um nível que torne explícito o que deve ser transferido ao software; que sirva como um contrato do que especificamente está incluído no projeto, considerando seu prazo e investimentos associados. Em abordagens com maior orientação à mudança, isso é diferente. O papel das histórias do usuário deve ser promover a negociação sobre o escopo de determinada iteração de desenvolvimento. As histórias do usuário devem manifestar objetivos de pequena granularidade que permitam a estimativa do investimento na entrega do software correspondente sem perder uma visão dos resultados associados. De posse desses dois parâmetros – resultados e investimentos –, é possível negociar trocas, escolher entre as limitações de tempo da próxima iteração e, dentre as várias histórias pendentes, escolher aquelas que serão alocadas para a próxima iteração.
V	Valioso (<i>Valuable</i>)	Software é como uma cebola; mas não porque nos faz chorar. O núcleo dessa cebola encosta no hardware, e sucessivas camadas com serviços específicos vão se acumulando até o limite externo dessa cebola: o usuário. A intenção é que determinado comportamento compartilhado esteja disponível em um único lugar e sem redundâncias desnecessárias. O “valioso” nesse contexto indica que uma história do usuário se refere à entrega de software que corresponda a um resultado na visão desse usuário e não em uma perspectiva técnica que descreve como serviços compartilhados são estruturados em diferentes camadas de uma infraestrutura comum de suporte. A história do usuário, portanto, não deve se referir a uma sub-rotina, um componente reutilizável.
E	Estimável (<i>Estimable</i>)	Estimar o esforço para desenvolver um produto de software como um todo de maneira direta – sem qualquer parâmetro – é um desafio quase impossível. O mesmo não deve acontecer com uma história do usuário. Soluções para estimativas como dias ideais (<i>ideal days</i>) ou pontos de história (<i>story points</i>) têm por objetivo atribuir um peso relativo entre as histórias. Essas referências então são utilizadas para determinar quais são as histórias do usuário que irão compor a próxima iteração, conforme a sua prioridade.
S	Pequeno	As histórias do usuário deveriam ser pequenas o suficiente de modo a

	(Small)	serem concluídas em uma interação. No desenvolvimento ágil, histórias do usuário originalmente estabelecidas no nível do objetivo do usuário podem ser decompostas em histórias de menor nível de maneira que estas possam atender a esse objetivo, apesar de, em um nível de granularidade mais fino, ainda observar os demais.
T	Testável (Testable)	A história do usuário deve ser passível de teste e, portanto, deve evitar terminologia que não seja clara.

8.8.2.2. CCC

Outro critério de qualidade para orientar na elaboração de histórias do usuário é a sua aderência a três características (CCC): ➤

Cartão: a história do usuário é pequena o suficiente para caber em um cartão.



Conversação: ela consegue promover a comunicação entre o usuário e o time, proporcionando um entendimento comum da funcionalidade a ser entregue.



Confirmação: associada à história do usuário, consta qual o comportamento esperado para confirmar o seu escopo. Também conhecido como plano de teste, escrito no verso do cartão.

Cada história do usuário deve ter em algum momento provas de validação associadas, permitindo que o desenvolvedor e, mais tarde, o cliente possam verificar se a história está completa. Como não se tem uma formulação completa dos requisitos funcionais e não funcionais, a falta de provas de validação abre a possibilidade de discussões longas e não construtivas no momento da entrega do produto.

Alguns exemplos de histórias do usuário são: ➤

Como operador do hotel, eu quero estabelecer taxas ótimas para os quartos no meu hotel para maximizar meus ingressos.

□

Como vice-presidente de marketing e vendas, eu quero rever o desempenho histórico de vendas, para identificar regiões geográficas e produtos de melhor desempenho.

□

Como cliente, quero consultar o catálogo, para que eu possa encontrar o produto que desejo comprar.

□

Como um cliente, quero que o produto seja acompanhado com o valor de desconto para compra à vista, para que eu tenha a visibilidade da diferença monetária do produto à vista ou a prazo.

□

Como um cliente, quero que os produtos selecionados para compra fiquem armazenados em um carrinho de compras, para que eu possa visualizar todos os meus produtos e o preço total.

8.8.2.3. Dividindo histórias do usuário

Antes de uma história do usuário estar pronta para ser agendada para execução em uma próxima iteração, ela deve ser “pequena o suficiente” para poder ser concluída dentro da iteração.

No entanto, muitas histórias do usuário começam maiores do que isso. Dividir uma história do usuário consiste em quebrá-la em partes menores, preservando a propriedade de que cada história do usuário deve ter, separadamente, valor de negócio mensurável.

Dean Leffingwell (2011) define dez padrões para dividir uma história do usuário, conforme apresentado na tabela a seguir.

Tabela 8.5. Padrões para simplificar dividindo histórias do usuário.

1. Etapas de fluxo de trabalho: procure por etapas no fluxo de trabalho que encadeiam diferentes papéis juntos ou diferentes funções que podem ser feitas de forma independente.
--

Como um gerenciador de conteúdo, posso publicar uma notícia para o site corporativo.	... eu posso publicar uma notícia diretamente para o site corporativo. ... eu posso publicar uma notícia com avaliação do editor. ... eu posso publicar uma notícia com revisão legal. ... eu posso ver uma notícia em um site de teste. ... eu posso publicar uma notícia do site de preparação para a produção.
2. Variações de regras de negócio: divisão das histórias para lidar com a complexidade das regras de negócio.	
Como usuário, eu posso procurar voos com datas flexíveis.	... eu posso encontrar voos para uma semana específica. ... eu posso encontrar voos entre datas específicas.
3. Maior esforço: a história pode ser dividida em várias partes para que o esforço recaia em implementar a primeira história. Adicionar mais recursos deveria ser relativamente trivial.	
Como usuário, posso pagar com cartão Visa, American Express, MasterCard.	... posso pagar com um tipo de cartão. ... posso pagar com todos os tipos de cartões.
4. Simples/complexo: procure por histórias gerais que escondem a complexidade. Quando a definição dos critérios de aceitação revela variadas maneiras de abordar isso, então você pode dividir ao longo dessa variância.	
Como um candidato a empréstimo, eu quero calcular meus pagamentos de hipoteca.	... calcular os pagamentos manualmente. ... utilizar um modelo de planilha <i>on-line</i> usar uma calculadora <i>on-line</i> .
5. As variações de dados: escolha objetos de dados que podem ter variações com base em funções e ações.	
Como tutor de cursos <i>on-line</i> , posso criar conteúdo.	... em espanhol. ... em inglês. ... em português.
6. Métodos de entrada de dados: algumas vezes a complexidade se encontra na interface de usuário e não na própria funcionalidade. O correto seria então dividir a história para criá-la com a interface mais simples e depois criar uma interface mais complexa.	

Como usuário, posso agendar data para viagem.	... fazendo uso de uma entrada única de dados. ... com uma interface de calendário.
7. Adiar qualidades do sistema: procure oportunidades para adiar o trabalho. Dividir a história entre o que eu preciso fazer para que funcione e o que eu preciso fazer para que seja mais rápido.	
Como usuário, eu posso pesquisar voos.	... a resposta é imediata. ... a resposta leva cinco segundos.
8. Operações (exemplo: Criar, Ler, Atualizar, Excluir – CRUD): concentre-se ao longo de operações (pense métodos de alto nível ou operações do tipo CRUD).	
Como dono de um supermercado, eu posso administrar produtos.	... eu posso adicionar produtos. ... eu posso atualizar dados de produtos. ... eu posso excluir produtos.
9. Cenários de casos de uso: se temos quaisquer cenários de casos de uso existentes para o sistema, podemos escrever e dividir histórias de acordo com os cenários individuais desses casos de uso.	
10. Quebre para fora um ponto: muitas vezes as histórias não são necessariamente complexas, mas apenas possuem muitas incógnitas. Transforme possíveis critérios de aceitação em perguntas e busque as respostas.	
Transforme “Como um vendedor, eu quero recolher o pagamento com PayPal, pois, como é universalmente aceito, vai aumentar minha receita” em... “Como funciona o PayPal?”	... como é que eu sei que tenho um pagamento bem-sucedido? ... como eu sei quando um pagamento não foi bem-sucedido e quais são as opções que eu posso apresentar para o comprador?

8.8.2.4. Épicos

Enquanto uma história do usuário no nível do objetivo do usuário pode ser objeto de subdivisão para que as histórias resultantes sejam pequenas o suficiente para que possam individualmente ser acomodadas a uma arquitetura de software e implementadas dentro de uma iteração, há manifestações do usuário que correspondem a objetivos agregadores. São os chamados “épicos”. Um exemplo de épico seria: *Como um operador de hotel, eu quero definir a taxa ideal para quartos no meu hotel.*

Observam-se vários objetivos do usuário nessa declaração. Então, pode-se dividir esse épico em diferentes histórias, como: a)

Como um operador de hotel, eu quero definir a taxa ideal para quartos com base em preços do ano anterior.

- b) Como um operador de hotel, eu quero definir a taxa ideal para quartos com base em quanto hotéis comparáveis ao meu estão cobrando.

8.8.2.5. Temas

Outro conceito relevante é tema. Um tema é uma coleção de histórias do usuário relacionadas. Por exemplo, histórias do usuário que lidam com o processo de registro de cursos para estudantes.

8.9. Técnica: modelagem de processos

O objetivo deste tópico não é ensinar modelagem de processos. É apresentar a informação básica para orientar o analista na leitura e interpretação de uma representação de processos existente ou para organizar requisitos alocando-os em processos de negócio em diferentes níveis.

8.9.1. O que é um processo?

Processo é um conjunto de atividades interdependentes, ordenadas no tempo e espaço de forma encadeada, que ocorrem como resposta a eventos e que possui um objetivo, início, fim, entradas e saídas bem definidos. Essas atividades são geralmente entre diferentes funções – tipicamente, representadas por unidades organizacionais, torres ou verticais dentro de uma mesma organização – ou entre organizações que trabalham juntas para criar um produto ou serviço final. As atividades são apresentadas no contexto da sua relação entre si para proporcionar uma visão da sequência e do fluxo. Essa visão inclui um conjunto definido de atividades ou outras ações realizadas por pessoas, sistemas ou uma combinação dos dois e tem um ou mais resultados que podem levar ao fim do processo ou a uma entrega de resultados para outro processo.

8.9.2. O que é modelagem de processo?

De acordo com o Guia para o Gerenciamento de Processos de Negócio (CBOK®), da Associação de Profissionais de Processos de Negócio (ABPMP), a modelagem de processos de negócio é o conjunto de atividades envolvidas na criação de representações de processos de negócio existentes ou propostos. Ela pode prover uma perspectiva ponta a ponta ou uma porção dos processos primários, de suporte ou de gerenciamento. O propósito da modelagem é criar uma representação do funcionamento do processo de maneira completa e precisa. Por esse motivo, o nível de detalhamento e o tipo específico de representação do processo têm como base o que é esperado da iniciativa de modelagem. Um diagrama simples pode ser suficiente em alguns casos, enquanto um modelo completo e detalhado pode ser necessário em outros.

Não se deve confundir modelagem de processo de negócio com gerência de processo de negócio. Ambos têm um acrônimo em inglês de BPM; contudo, este termo é aplicado para o último: *Business Process Management*.

Praticamente todas as organizações criam descrições de seus principais processos em diferentes níveis de detalhe. Descrevem, por exemplo, como enviar as remessas de um cliente ou como gerar os relatórios financeiros de fechamento de mês. Em muitos casos, vão além de descrições textuais, incluindo modelos gráficos, que normalmente são fluxogramas (ou uma de suas muitas derivações) que determinam de que forma as atividades estão inter-relacionadas.

A modelagem de processos de negócio permite criar uma abstração de como funciona um negócio, pois fornece o entendimento de como são realizadas as diversas atividades contidas em cada processo. Na modelagem de processos, informações e documentos são utilizados pelos analistas, gerando um fluxo de como as atividades são realizadas, desde seu início até alcançar o objetivo do processo.

8.9.3. Diagrama, mapa e modelo de processos

Os termos “diagrama de processo”, “mapa de processo” e “modelo de processos” são muitas vezes utilizados de forma intercambiável ou como

sinônimos. Contudo, diagramas, mapas e modelos têm diferentes propósitos e aplicações. Na prática, diagrama, mapa e modelo são diferentes estágios do desenvolvimento do processo, cada qual agregando mais informação e utilidade para entendimento, análise e desenho de processos: ➤

Um **diagrama** retrata os principais elementos de um fluxo de processo, mas omite detalhes menores de entendimento dos fluxos de trabalho. Uma analogia pode ser feita com um diagrama simples que pode ser utilizado para demonstrar a rota até um local de armazenagem; ele pode retratar coisas como marcos geográficos e distâncias de forma simplificada ou exagerada, mas ainda assim serve para ajudar a encontrar o armazém. De maneira similar, um diagrama de processo ajuda rapidamente a identificar e entender as principais atividades do processo.



Um **mapa** fornece uma visão abrangente dos principais componentes do processo e apresenta maior precisão do que um diagrama. Tenderá a agregar maior detalhe acerca do processo e de alguns dos relacionamentos mais importantes com outros elementos, tais como atores, eventos e resultados.



Um **modelo** implica na representação de um determinado estado do negócio (atual ou futuro) e dos respectivos recursos envolvidos, tais como pessoas, informação, instalações, automação, finanças e insumos. Como é utilizado para representar com mais precisão o funcionamento daquilo que está sendo modelado, requer mais dados acerca do processo e dos fatores que afetam seu comportamento.

Para que os modelos de processo cumpram esse propósito, recomenda-se a utilização de padrões. A *Object Management Group* (OMG) estabelece o diagrama de atividades como parte da *Universal Modeling Language* (UML) e que pode ser usado também para modelagem de processos de negócio. A mesma organização também define um padrão especificamente para esse fim denominado *Business Process Model and Notation* (BPMN).

8.9.4. Por que modelagem de processos?

Uma vez criados, modelos de processo são documentos operacionais valiosos, que podem orientar funcionários nos passos a serem seguidos nos processos, garantindo que estes sejam desempenhados de forma padronizada e possibilitando que todas as partes interessadas tenham o mesmo conhecimento dos processos. Documentar processos permite que técnicas analíticas sejam aplicadas, manualmente ou por meio de software de análise de modelos, para detectar deficiências e congestionamentos nos processos e simular processos melhorados antes que estes sejam postos em prática.

Dentre alguns dos principais objetivos da modelagem de processos, podemos citar: ➤

Entender a estrutura e a dinâmica das áreas da organização.



Entender os problemas atuais da organização e identificar potenciais melhorias.



Assegurar que todos tenham entendimento comum da organização.



Servir como insumo para a Engenharia de Requisitos.



Auxiliar a identificação de competências.

8.9.5. Como realizar a modelagem de processos?

Modelos de processos podem ser criados manualmente ou com o uso de ferramentas específicas de diagramação; as mais sofisticadas são chamadas de *Business Process Management Suites* (BPMS). Um BPMS é um

conjunto de ferramentas automatizadas que proveem suporte ao BPM. Possibilita a modelagem, a execução (simulada ou não), o controle e o monitoramento dos processos de forma automatizada. Define a arquitetura e a infraestrutura tecnológica necessárias para a modelagem do negócio, a execução em produção dos fluxos de trabalho, a aplicação de regras de negócio, a utilização de dados corporativos, a simulação de cenários e a operação de outras aplicações do ambiente BPMS.

Na modelagem BPMN são construídos os diagramas de fluxo de trabalho (*workflow*), cada um contendo as atividades realizadas. Recomenda-se usar um verbo no infinitivo, seguido de um nome e algum detalhe, se necessário. Por exemplo, “enviar correspondência”. Todos os modelos devem estar ligados em um fluxo de como são realizadas as atividades. As etapas de cada processo devem ser descritas de forma que qualquer pessoa que leia o modelo consiga entender. O recomendado é que seja feito da forma mais simples e clara possível. Raias são utilizadas para que seja possível identificar qual ator realizou determinada atividade e quando houve a mudança para outro ator.

Os elementos da modelagem de processos de negócios são: ➤

Evento: acontecimento que inicia a execução (inicial), afeta o comportamento do processo (intermediário) ou o conclui (final).



Atividades: conjunto de ações realizadas.



Atores: responsáveis pelas atividades.



Entradas/saídas: insumos necessários para o processo ser executado na entrada e nas saídas geradas ao final do processo.



Regras: restrições que causam dependências entre atividades.

8.9.5.1. Representações de processos de negócio e a Engenharia de Requisitos

No contexto da Engenharia de Requisitos, quaisquer das representações citadas cumprem os propósitos abordados neste livro. A ideia é elevar-se a um nível superior ao de uma tarefa. Colocar em perspectiva diferentes níveis de objetivos de negócio para mais facilmente entender ou confirmar como os objetivos associados às diferentes tarefas se encadeiam nessa direção.

A modelagem de processo representa de maneira simples a dimensão colaborativa da solução. Isso faz com que todas as partes interessadas compartilhem a mesma visão (geralmente cada uma só conhece o seu papel no processo). Com isso, aumenta-se significativamente o potencial do conjunto de requisitos para a solução proposta estar alinhada ao negócio, cumprir o propósito de atender às suas necessidades e alcançar os resultados desejados.

Em função das responsabilidades do analista de requisitos, ele não utiliza a modelagem de processos para redesenhar o fluxo operacional de uma organização. Contudo, as diferentes representações de processos são instrumentos úteis para que sejam identificadas oportunidades de racionalização e eliminação de redundância ou informações em conflito. Apontar tais oportunidades e conflitos deve ser um dos objetivos da análise de requisitos, e a modelagem de processos tem papel central nisso.

Algumas outras aplicações práticas da modelagem de processos: ➤

Preparação para elicitação: um modelo de processo já existente permite descobrir quais papéis são desempenhados nos macroprocessos incluídos no escopo. Isso é relevante para descobrir as responsabilidades de partes interessadas já identificadas ou para descobrir a necessidade de identificar novas partes interessadas que desempenhem papéis. Também permite depreender um entendimento sobre como o seu trabalho é realizado, quais são os objetivos de nível mais alto e que produtos ou serviços são gerados.



Execução da elicitação: mapas, diagramas ou modelos de processo podem ser aproveitados para complementar entrevistas – atividades realizadas prioritariamente aos pares – em eventos de elicitação em grupo onde a representação escolhida é utilizada como fio condutor para confirmar ou retificar as informações, identificar e tentar resolver junto ao grupo conflitos presentes como ilustrado na Figura 8.11.



Modelagem de requisitos: ainda que não esteja disponível uma representação de processos, montar um fluxo descrevendo o processo a partir dos requisitos é uma experiência que traz resultados positivos similares.



Validação de requisitos: um modelo de processo é mais adequado ao uso de protótipos porque permite simular com precisão o papel dos requisitos identificados como parte da solução mais abrangente. Observe que não se deve associar a validação ao tempo de testes de aceitação do usuário ao final de projeto. Ela deve ser associada ao ponto em que há protótipos abordando uma área de processo e na qual se viabilize uma dinâmica similar à apresentada na Figura 8.11, mas substituindo requisitos por protótipos e uma experiência mais próxima ao uso.

Seja para descobrir requisitos ou para validar a sua eficácia, contribuindo para atender às necessidades de negócio, a modelagem de processos permite confirmar se uma atividade realmente deve se manter manual, se realmente parte ou todo o trabalho é responsabilidade de outro sistema, ou se houve falha na elicitação.

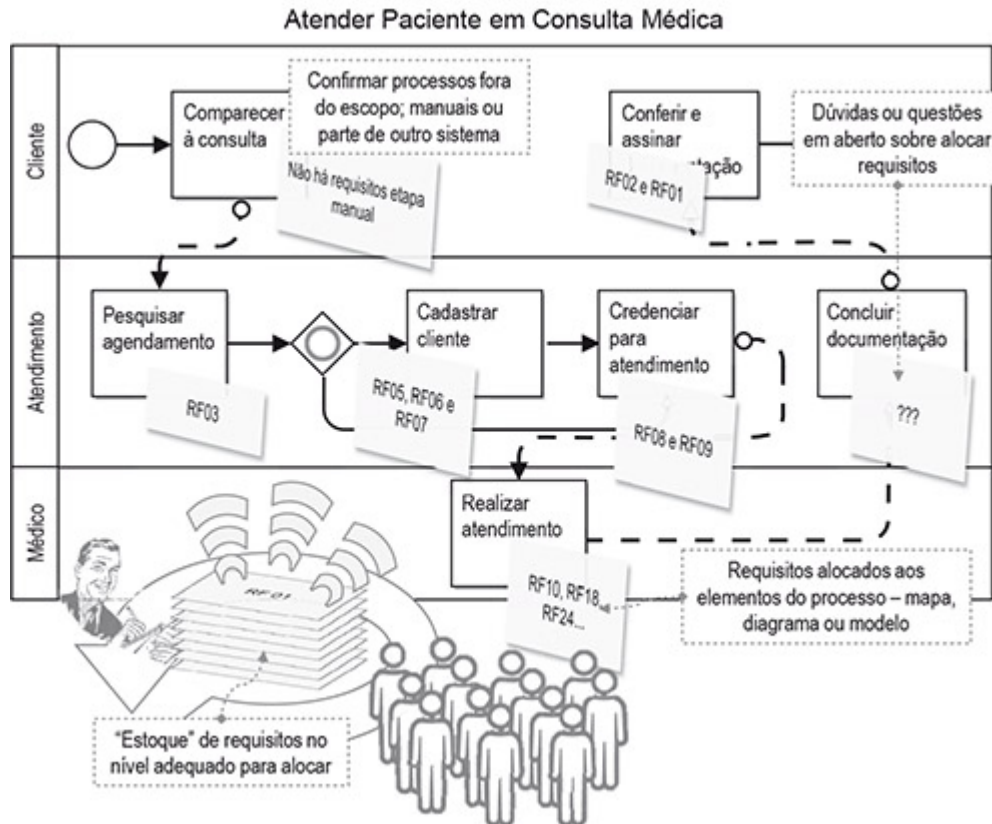


Figura 8.11. O modelo de processos e suas aplicações na Engenharia de Requisitos.

8.10. Técnica: decomposição funcional

Em termos práticos, qualquer coisa de maior complexidade é passível de ser decomposta em seus elementos constituintes mais simples. Por sua vez, esses elementos também podem ser decompostos. Essa dinâmica em sucessivas vezes constrói uma hierarquia com diferentes níveis de decomposição funcional. Isso se aplica aos requisitos, que também podem ser levantados e especificados em diferentes níveis de expansão.

Toda hierarquia requer ao menos dois elementos, um superior e outro subordinado a ele. Um elemento não deve ser subordinado a mais de um elemento superior. Um elemento superior deve ter ao menos um elemento subordinado, não havendo limite máximo para a quantidade de elementos subordinados. Em termos práticos, há limitações. Por exemplo, a “Lei de Miller” (MILLER, 1956) descreve a limitação da capacidade da mente humana em processar informação. Ela coloca que a quantidade de objetos

que alguém consegue manter em memória de maneira simultânea está em sete, mais ou menos dois. Os elementos subordinados, ainda que únicos, compartilham aspectos em comum de tal forma que haja afinidade suficiente entre os elementos pares para estarem subordinados ao mesmo elemento superior. A Figura 8.12 representa esses conceitos e ilustra a sua inter-relação.

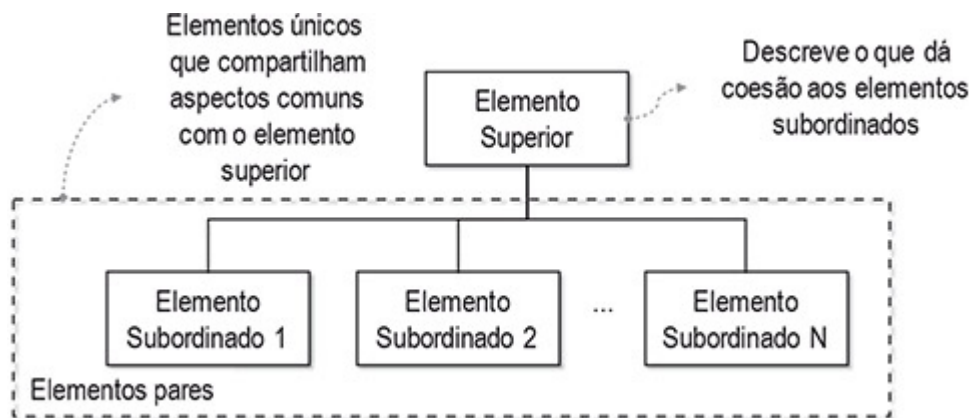


Figura 8.12. A hierarquia resultante da decomposição funcional, seus elementos e inter-relação.

Os aspectos comuns no caso dos requisitos organizados em uma hierarquia estão relacionados à sua especialização qualificada pelo elemento superior.

Por exemplo, durante a elicitação de requisitos para o desenvolvimento do PowerPoint identificam-se funcionalidades de inserir uma forma, definir o seu estilo, escolher como será o seu preenchimento, escolher como será o seu contorno, aplicar a formatação de outra forma já existente *etc.* Todas essas tarefas subordinam-se a ações sobre a forma (o elemento “Formatar – Ferramentas de Desenho”, na Figura 8.13).

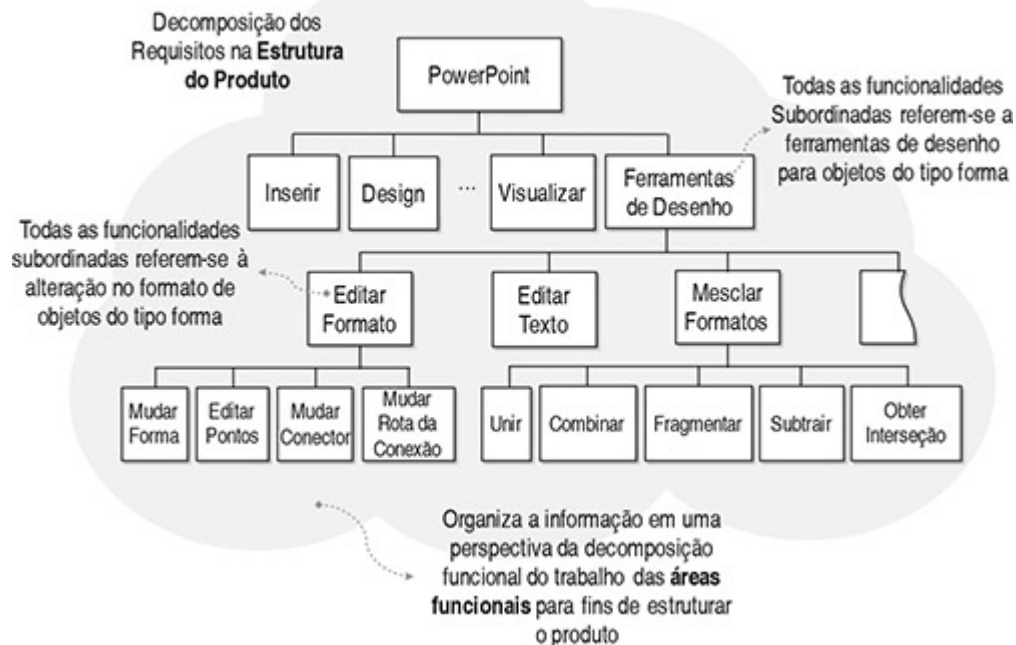


Figura 8.13. Uma visão de decomposição funcional do PowerPoint, destacando macrofuncionalidades que definem as características comuns das funcionalidades subordinadas.

Um elemento análogo à decomposição funcional e muito usado em planos de projeto é a estrutura analítica de projeto (EAP ou *Work Breakdown Structure* – WBS), cujo objetivo é lidar com vários elementos de menor complexidade em vez de um único elemento de maior complexidade e eliminar incertezas quanto ao que mais especificamente deve ser feito. A diferença é que a EAP estabelece uma hierarquia de atividades do projeto (ou pacotes de trabalho). Já a decomposição funcional dos requisitos estabelece uma hierarquia de atividades até o nível das tarefas do usuário que serão oferecidas pelo produto de software. A Figura 8.14 coloca essas duas visões em perspectiva.

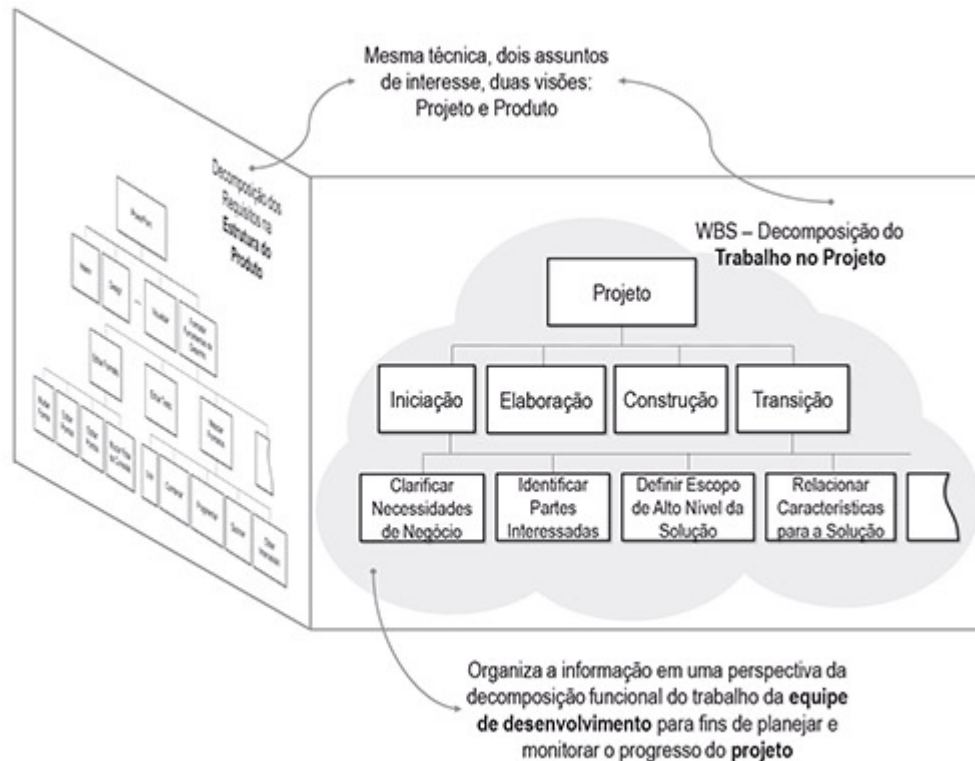


Figura 8.14. A mesma técnica, dois assuntos diferentes estruturando o produto e o projeto.

8.10.1. Como aplicar a decomposição funcional?

A decomposição funcional na análise de requisitos é um meio para organizar e dar estrutura aos requisitos; não uma estratégia para o seu desenvolvimento.

O nível apropriado de decomposição funcional define onde, por que e como parar a decomposição do assunto de forma a atender aos objetivos da análise. Como o nível de tarefa é o único que pode ser identificado de maneira inequívoca, recomenda-se este patamar como ponto no qual há entendimento e detalhe suficientes para que seja possível usar os resultados da decomposição funcional em outras tarefas, como: a verificação de requisitos, a validação de requisitos e a especificação de requisitos.

8.10.2. Por que aplicar a decomposição funcional?

O ponto de partida para a elaboração da especificação são as necessidades de negócio. A partir delas as informações solicitadas são refinadas junto às partes interessadas de forma a descobrir as tarefas ocultas em requisitos funcionais agregadores.

Portanto, o uso dos conceitos associados à decomposição funcional está muito mais próximo a uma “composição funcional”, onde o estoque de requisitos funcionais (já refinados e identificados no nível de tarefa, no nível de objetivo do usuário) é agregado em uma hierarquia com diferentes níveis de forma a facilitar outras atividades. A Figura 8.15 ilustra essa dinâmica.

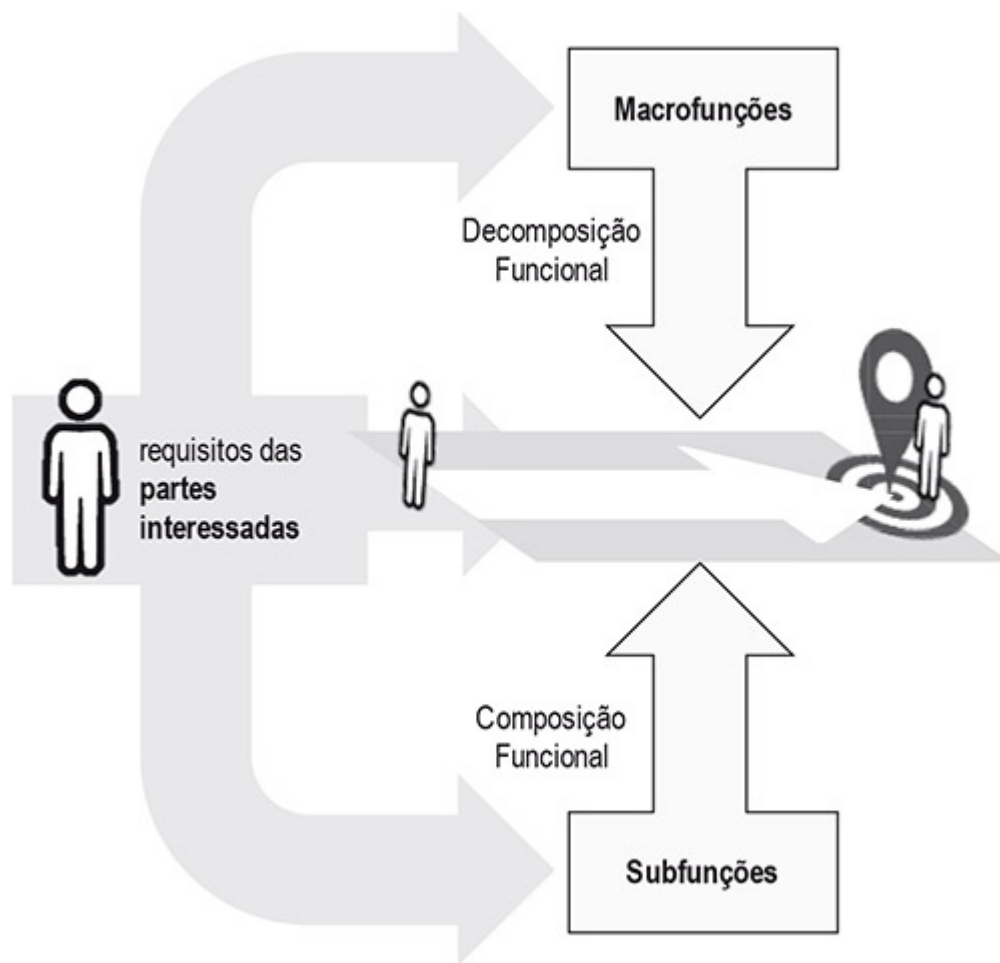


Figura 8.15. A dinâmica de composição e decomposição da informação.

As informações advindas dessa dinâmica de composição e decomposição funcional têm um papel importante na elicitação. Eventos de elicitação junto a grupos têm uma dinâmica diferente de uma entrevista. Não costuma

ser produtivo realizar um *workshop* de requisitos sem que na preparação prévia haja um fio condutor para apresentar as descobertas já realizadas. O papel dos requisitos organizados em uma hierarquia de decomposição funcional é solicitar retroalimentação sobre áreas que precisam de mais aprofundamento.

Outro benefício da decomposição funcional é evidenciar de maneira visual e de fácil percepção partes “fracas” do escopo. Ou seja, a figura final da decomposição, quando apresentada de forma não harmônica (por exemplo, partes muito mais profundas que outras), pode indicar que não foi dada a devida atenção a essas partes.

8.11. Técnica: modelagem de domínio

A modelagem de domínio é uma extensão da elaboração do glossário. Ela busca identificar conceitos do negócio sobre os quais há necessidade de recuperação ou armazenamento de dados e as suas inter-relações com outros conceitos no domínio do problema.

8.11.1. Como realizar a modelagem de domínio?

O modelo de domínio deve ser elaborado a partir do entendimento dos elementos que compõem o domínio do problema. Isso é feito por meio da identificação dos conceitos de negócio nos quais dados são consultados ou armazenados pelo software.

É indicado começar a modelagem de domínio em momentos iniciais do desenvolvimento ou mesmo antes dele. Nesses momentos, ainda não se sabe quais são os requisitos funcionais e quais deles serão alocados ao software em desenvolvimento. Portanto, inicialmente, a modelagem de domínio não deve se limitar aos requisitos com a visão da solução, pois eles ainda não estão completamente desenvolvidos.

Versões iniciais do modelo de domínio devem ultrapassar um limite ainda incerto; portanto, elas não devem excluir conceitos de negócio com potencial de alocação ao escopo final dos produtos em desenvolvimento, dependendo de decisões ainda em aberto do projeto.

Os principais elementos de um modelo de domínio são os conceitos de negócio e os relacionamentos entre eles. Ambos são descritos a seguir.

8.11.2. Conceito de negócio

Um conceito de negócio pode ser qualquer coisa que tenha um significado para o negócio. É uma construção que representa como uma unidade coesa um grupo de dados relativos a um mesmo assunto. As necessidades de informação associadas ao funcionamento do negócio é o que promove essa unidade. Essa coesão é denominada dependência funcional.

Por exemplo, um sistema informatizado para suporte à gestão da contratação de serviços de TI desde o seu planejamento até a sua avaliação e encerramento aderente à IN-04 do (MPOG, 2014). A norma estabelece: *Subseção I – Da instituição da Equipe de Planejamento da Contratação*
Art. 11. A fase de Planejamento da Contratação terá início com o recebimento pela Área de Tecnologia da Informação do Documento de Oficialização da Demanda – DOD, a cargo da Área Requisitante da Solução, para instituição da Equipe de Planejamento da Contratação, que conterà no mínimo: I – necessidade da contratação, considerando os objetivos estratégicos e as necessidades corporativas da instituição, bem como o seu alinhamento ao PDTI; II – explicitação da motivação e demonstrativo de resultados a serem alcançados com a contratação da Solução de Tecnologia da Informação; III – indicação da fonte dos recursos para a contratação; e IV – indicação do Integrante Requisitante para composição da Equipe de Planejamento da Contratação.

Em momentos iniciais do desenvolvimento, todos os substantivos e nomes deverbais (mais detalhes adiante) devem ser avaliados como candidatos a conceitos de negócio para compor um modelo de domínio. Esses tipos de termos estão sublinhados no texto exemplo.

Os substantivos possivelmente representam objetos de interesse, como documentos, pessoas, estruturas organizacionais, parâmetros operacionais ou estruturas físicas. Esses objetos de interesse possuem estrutura, comportamento ou inter-relações como qualidades.

A solução pode necessitar armazenar ou recuperar informação que descreve essas qualidades. O exercício da Engenharia de Requisitos transformará esse “pode” em um “deve” conforme cada caso.

Nome deverbal é a forma nominal de um verbo – por exemplo, *receber* é o verbo e *recebimento* é sua forma nominal. Possivelmente representam eventos ou manifestações que também possuem as mesmas qualidades citadas e o mesmo potencial para armazenamento ou recuperação pelo software.

Deve-se filtrar logo de início os candidatos que não refletem um conceito de negócio. Por exemplo, na expressão “explicitação da motivação” o autor apenas quer indicar um único conceito, “motivação”, e não sugere necessidade de armazenar dados sobre um evento que torna a motivação explícita. É um caso similar à “descrição da motivação”, que não indica um marco no fluxo operacional especificamente para descrever a motivação.

Já na expressão “Indicação do integrante”, há um conceito de negócio com informações sobre um rito (“a indicação”) com várias necessidades de informação possíveis, como: ➤

Quando da indicação?

☐

Indicação de quem?

☐

Qual o objetivo da indicação?

☐

Por quanto tempo é a indicação?

Há também outro conceito de negócio sobre “o integrante”, com necessidades de informação como: ➤

Quem é ele?

☐

Como se pode entrar em contato com ele?

Para a modelagem de domínio, o interesse são conceitos de negócio que agreguem um grupo de dados funcionalmente dependente. Então, a “explicitação da motivação” nem cabe representar no modelo como um conceito de negócio, pois não há qualquer informação que dependa desse conceito. Ela é parte de um grupo de dados sobre o conceito de “Documento de Oficialização de Demanda”. Na dúvida, confirme com as partes interessadas.

Algum conhecimento sobre o domínio do problema pode eliminar, ou ao menos diminuir, a prioridade dos candidatos que, já se sabe, estarão fora do escopo da solução e estão presentes apenas para dar contexto. Ainda assim, lembre-se de que você não é a autoridade no assunto e confirme com a parte interessada.

Quais requisitos de armazenamento serão derivados desses conceitos de negócio são decisões ainda pendentes ao longo do desenvolvimento dos requisitos. O analista deve investigar quais desses conceitos de negócio devem ter dados mantidos ou recuperados pelo software para que os objetivos de negócio que motivaram o projeto sejam alcançados. Essa informação é alcançada por meio de atividades de elicitação.

Uma primeira versão de um modelo de domínio poderia ser como a ilustrada na Figura 8.16. Sua elaboração promove o desenvolvimento de pauta para atividades de elicitação, por exemplo: ➤

Sobre a motivação e os resultados: há alguma informação além de sua descrição? Há padrão com categorias complementares à sua descrição específica para a contratação? Há realmente uma descrição específica para contratação ou apenas categorias nas quais cada contratação é enquadrada?

□

Sobre os objetivos estratégicos, o PDTI, as necessidades corporativas, as fontes de recursos: são todos resultados de decisões anteriores à contratação? Há registro prévio com as informações necessárias sobre esses conceitos no negócio? Esses

conceitos de negócio devem ser mantidos independentemente da contratação? O que se precisa saber especificamente sobre cada um deles?



Sobre a indicação da fonte de recursos, a instituição da equipe de planejamento, a indicação do integrante requisitante: existe um evento no negócio relativo a esses eventos na contratação e sobre o qual se registram informações como o responsável por ele? Por força de qual instrumento decisório ele aconteceu? Qual o limite em sua utilização etc.? Quando se colocam esses eventos, referem-se apenas a uma descrição que designe recursos humanos e materiais para a contratação?

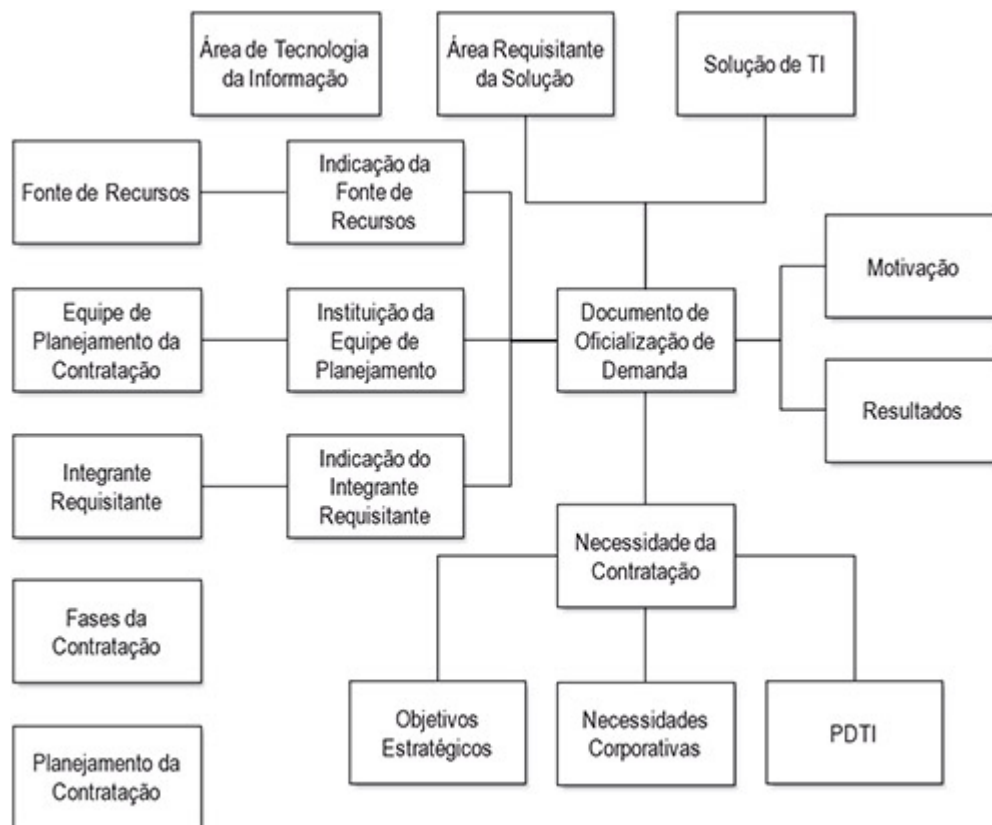


Figura 8.16. Versão inicial do modelo de domínio para o trecho apresentado da IN-04.

Conforme são obtidas respostas às perguntas, o modelo vai sendo aperfeiçoado. Por exemplo, se as respostas indicarem que: ➤

não há nada além de uma descrição para registrar no Documento de Oficialização de Demandas a sua motivação, os resultados esperados e as necessidades corporativas, esses elementos devem ser removidos como conceitos de negócio do modelo; ➤

os objetivos estratégicos têm uma série de informações associadas; essas informações são mantidas pela Secretaria de Planejamento; e o Documento de Oficialização de Demanda apenas precisa referenciar uma Ação Programática no Planejamento Estratégico; deve-se manter no modelo de domínio um conceito de negócio referente a essas informações.

As necessidades de informação típicas, que motivam requisitos de armazenamento sobre os conceitos de negócio identificados, são relacionadas à descrição de: ➤

Sua estrutura: qual a estrutura de um Documento de Oficialização de Demanda? Que informações ele contém?



Seu comportamento: quem elabora o Documento de Oficialização de Demanda? Qual o seu ciclo de vida? Quais etapas desse ciclo de vida devem ter tarefas total ou parcialmente transferidas para o software?



Suas inter-relações: há relação entre o Documento de Oficialização de Demanda e a Área Requisitante da Solução? Que outras relações relevantes existem?

A modelagem de domínio permite organizar os requisitos das partes interessadas e os requisitos de negócio em requisitos passíveis de serem verificados. A meta deve ser esgotar o escopo da solução proposta em termos de sua capacidade para: ➤

capturar, validar e complementar dados recebidos do ambiente externo sobre esses conceitos de negócio; ➤
consultar a situação de conceitos de negócio internos ou externos à organização; ➤
fornecer informações e resultados a partir dos dados armazenados sobre esses conceitos de negócio.

Não devem ser consideradas apenas as necessidades de pessoas ao investigar e organizar as necessidades de informação sobre conceitos de negócio. Devem ser levadas em conta também as necessidades de integração da solução em desenvolvimento com outras soluções, equipamentos ou qualquer tipo de entidade que consuma ou forneça informações para a solução proposta.

8.11.3. *Checklist* para organizar o modelo de domínio

São assuntos que devem ser usados como referência para identificar os conceitos de negócio: ➤

Documentos tramitados e cujos dados são informados, transformados, armazenados e distribuídos em seu teor bruto ou com informações derivadas a partir deles.



Pessoas representando os diferentes papéis que necessitam interagir com o software.



Eventos que demarcam diferentes pontos do processo de negócio.



Manifestações que provocam a ação e o acompanhamento dessa ação por meio da solução em desenvolvimento.



Estruturas organizacionais com suas alçadas e competências no âmbito do software. Por exemplo: filiais, departamentos, gerências, lojas, agências, diretorias.



Parâmetros operacionais que limitam e estabelecem políticas que devem ser respeitadas pelo software. Praticamente todos os sistemas possuem parâmetros.



Estruturas físicas com a sua descrição e qualificação, como prédios, salas, plantas, fábricas, produtos, depósitos, localizações, equipamentos *etc.*

8.11.4. Relacionamentos entre conceitos

Os relacionamentos entre os conceitos de negócio são informações relevantes porque a análise dos requisitos das partes interessadas normalmente explica as regras de alguns relacionamentos e deixam outros em aberto.

Nem sempre essas lacunas de informação são facilmente percebidas de outra forma. A atualização do modelo de domínio facilita identificá-los de maneira a promover a elaboração de questões para superá-los.

Por exemplo, um mesmo objetivo estratégico pode ter nenhuma contratação para que seja alcançado, uma ou várias. Uma contratação deve estar associada ao menos a um objetivo estratégico, mas pode também estar associada a vários.

8.11.5. Representações para o modelo de domínio

Uma representação útil para indicar o relacionamento entre diferentes conceitos de negócio é utilizada na modelagem de dados; destacando que o objetivo não é produzir um modelo de dados para implementar em um banco de dados ou mesmo um modelo lógico observando regras de

normalização. O objetivo é produzir um modelo de dados conceitual que descreva o domínio do problema e que, durante a sua elaboração, o analista possa mais facilmente descobrir o que ainda não se sabe.

Um modelo de classes no nível de conceitual também pode ser utilizado com o objetivo de representar um modelo de domínio.

A Figura 8.17 ilustra a representação de “pés de galinhas” e a representação textual das regras que definem as quantidades mínimas e máximas entre casos de conceitos de negócio. Por exemplo, uma mãe biológica deve ter ao menos um filho (caso contrário, não seria uma mãe) e pode ter vários (a mulher com maior número de filhos de que se tem notícia teve 69 filhos). Cardinalidade é o nome que se dá ao que essas regras descrevem.

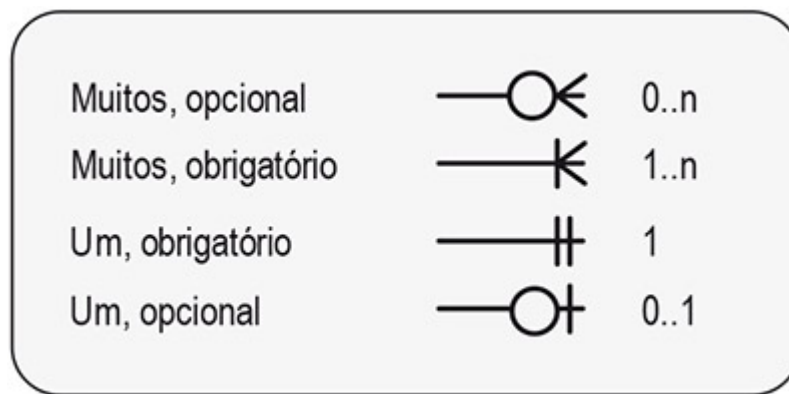


Figura 8.17. A representação “pés de galinha” e a representação da cardinalidade para os relacionamentos entre os conceitos de negócio.

Por exemplo, após mais investigação, descobriu-se mais informação sobre os relacionamentos entre os conceitos de negócio e as regras que definem a sua estrutura; com a nova informação, o modelo de domínio evoluiu. A Figura 8.18 ilustra como ficou uma parte do modelo sob a luz das novas informações.

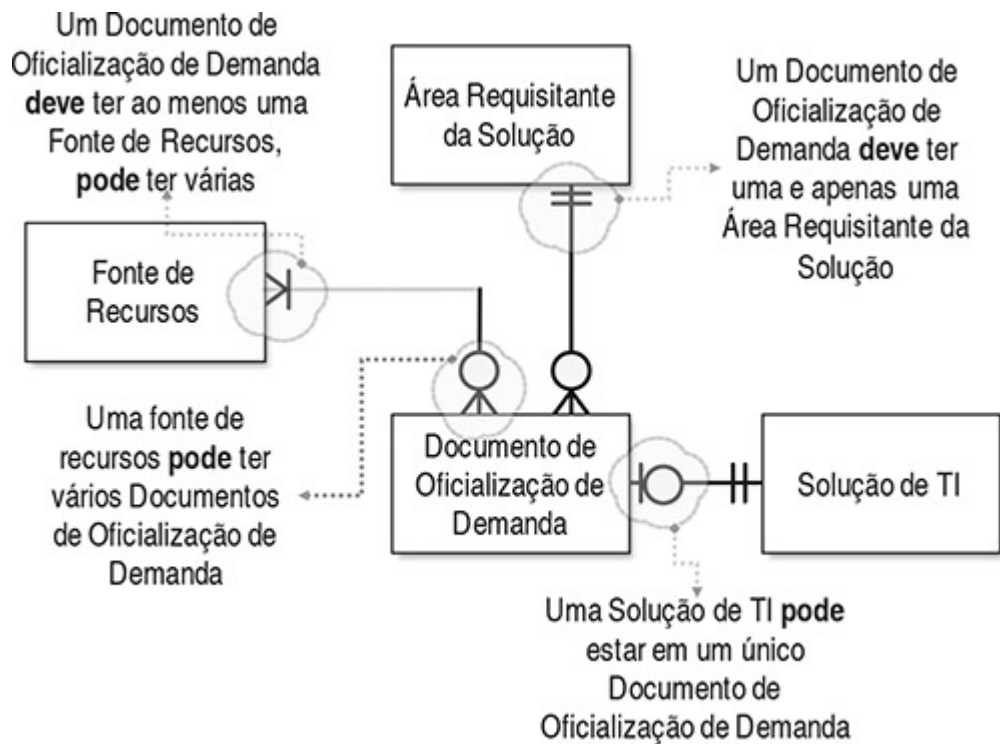


Figura 8.18. Fragmento do modelo de domínio com a representação de regras que explicam a relação entre os diferentes conceitos de negócio.

8.11.6. Por que realizar modelagem de domínio?

A modelagem de domínio fornece uma visão estrutural dos requisitos. Isso complementa a visão colaborativa fornecida pela modelagem de processo e as visões em diferentes especialidades fornecidas pela decomposição funcional.

O objetivo da modelagem de domínio é organizar a informação obtida na elicitação de requisitos em conceitos de negócio e descrever as regras que determinam o relacionamento entre eles.

Priorize, na utilização em atividades em grupo junto às partes interessadas, produtos da modelagem de processos ou da decomposição funcional. Contudo, a mesma dinâmica utilizada na alocação de requisitos aos processos ou na organização dos requisitos em uma hierarquia pode ser aplicada ao modelo de domínio com os mesmos objetivos e resultados.

Não é objetivo da modelagem de domínio no âmbito da disciplina de requisitos produzir um modelo de dados que obedeça a regras de normalização. Nem tampouco identificar e descrever todos os atributos e os

dados dependentes dos conceitos de negócio identificados. Há atividades em outras disciplinas para tratar desses objetivos.

8.12. Técnica: modelagem de casos de uso

Todo software deve ter um comportamento em resposta a eventos promovidos por seus usuários ou promover eventos que façam esses usuários agirem. Uma solução para especificar o comportamento associado a essa funcionalidade é por meio da descrição de casos de uso do software pelos seus usuários, denominados atores na terminologia de quem define seu padrão, a UML.

8.12.1. O que é um caso de uso?

Um caso de uso é um conjunto de passos que descreve um cenário principal e possíveis cenários alternativos para um ator alcançar um objetivo com o uso do sistema.

Como todo requisito funcional, o caso de uso não entra no mérito de como o comportamento descrito será mapeado nos componentes em uma arquitetura de hardware e software, ou como este será implementado em uma linguagem de programação. Portanto, ele descreve o que o software deve fazer e não como ele deve fazê-lo (em uma perspectiva de *design* ou implementação).

A especificação de requisitos funcionais que utiliza casos de uso é composta por: ➤

- diagrama de casos de uso;



- descrição dos atores;



- especificação dos casos de uso.

8.12.2. Diagrama de casos de uso

O diagrama de casos de uso demonstra de forma gráfica quais funcionalidades do software atenderão a quais usuários específicos. Permite a identificação do caso de uso como uma unidade de função para o software em análise. Representa graficamente os casos de uso, os papéis que os usuários desempenham (denominados atores por esse motivo) e a inter-relação desses elementos. A Figura 8.19 ilustra um diagrama de casos de uso e seus elementos básicos são descritos a seguir: ➤

Ator: representa uma pessoa (ou um grupo de pessoas) que desempenha um papel ao interagir com o software. Pode ser também qualquer “coisa” que interage com o software com a finalidade de cumprir um objetivo. Exemplos desses outros atores são outros sistemas ou dispositivos. Seu símbolo é um boneco palito.



Caso de uso: representa uma funcionalidade que atende a um ou mais requisitos do cliente. Como nome, sugere-se usar um verbo no infinitivo com um complemento; por exemplo, “incluir cliente”. Seu símbolo é uma elipse com seu nome no interior. Os casos de uso podem opcionalmente estar envolvidos por um retângulo que representa os limites do sistema.



Relacionamento: um ator interage com um caso de uso e isso é representado por um relacionamento. Os casos de uso também podem se relacionar entre si. Seu símbolo é uma linha ou uma seta.

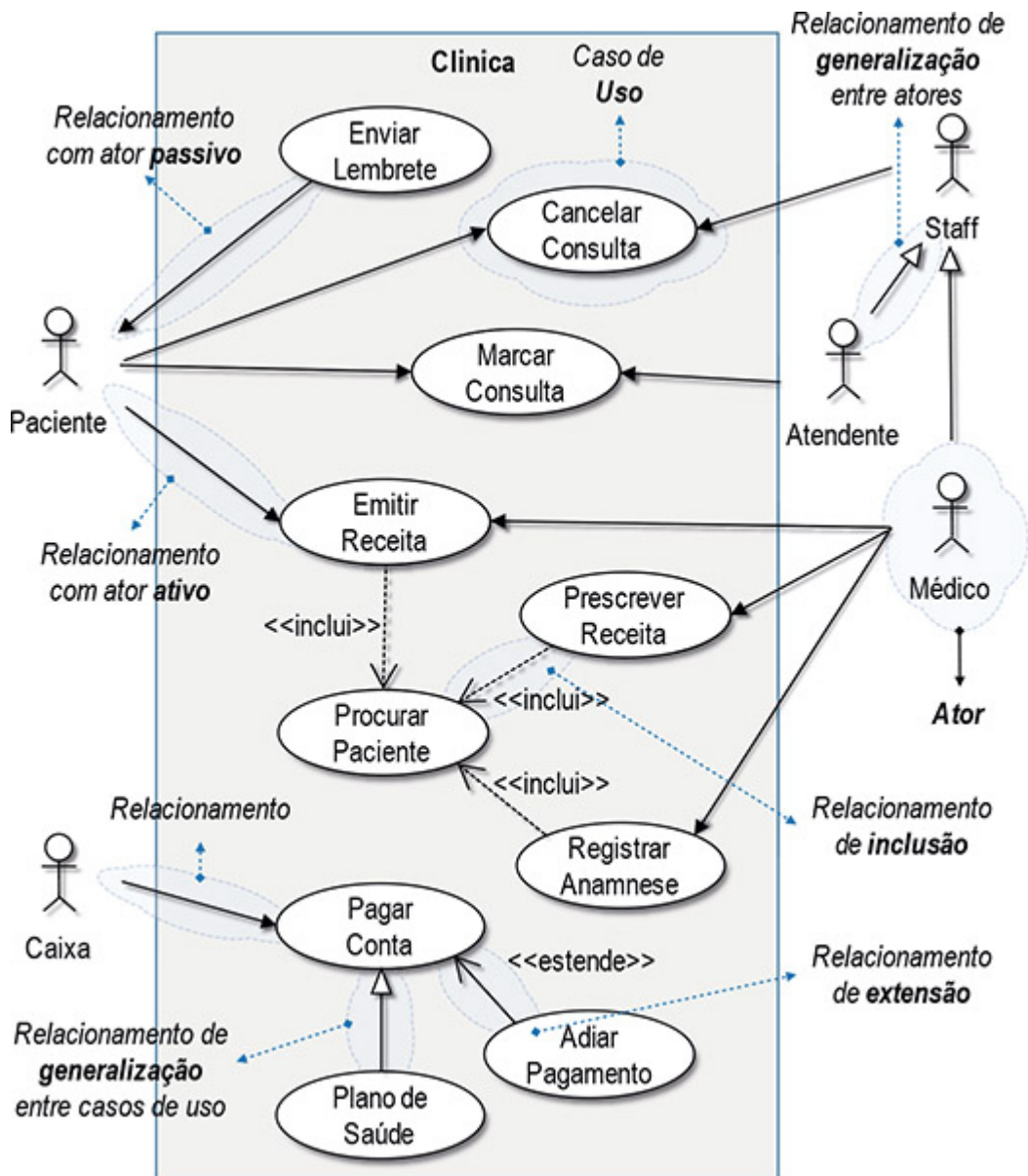


Figura 8.19. Diagrama de casos de uso e seus elementos.

8.12.2.1. Associação entre um ator e o caso de uso

A associação entre um ator e um caso de uso é denominada relacionamento de comunicação. Nessa relação, distinguem-se: ➤

Ator ativo: quando este inicia (ou dispara) a execução do caso de uso. A seta no relacionamento de comunicação aponta para o caso de uso, como no diagrama da Figura 8.19, onde o Paciente inicia o caso de uso “Marcar Consulta”. Quando a linha do relacionamento não possui seta, convencionam-se que o ator é ativo.



Ator passivo: quando o caso de uso não é iniciado pelo ator e ele reage a uma provocação do software que inicia a comunicação. A seta no relacionamento de comunicação aponta para o ator. Exemplos comuns são notificações e alarmes, como no diagrama da Figura 8.19, onde o Paciente é instado a agir pelo caso de uso “Enviar Lembrete”.

8.12.2.2. Outros tipos de relacionamento

Existem tipos especiais de relacionamentos entre os elementos de um diagrama de casos de uso. São eles: ➤

Generalização ou especialização.



Extensão.



Inclusão.

8.12.2.2.1. Generalização ou especialização A condição para identificar um caso de uso de generalização é haver um comportamento e uma finalidade comuns compartilhados por diferentes casos de uso que se especializam em um comportamento específico. O caso de uso de generalização tem o objetivo de prover uma descrição unificada desse comportamento comum em vez de tê-lo descrito várias vezes. Ao estabelecer um relacionamento entre um caso de uso de comportamento genérico e outro que se especializa nesse comportamento, diz-se que este último herda o comportamento descrito pelo primeiro.

No diagrama esse tipo de relacionamento é representado por uma seta de ponta fechada partindo do caso de uso de especialização em direção ao caso de uso de generalização.

8.12.2.2.2. Extensão O caso de uso estendido representa um comportamento opcional que o caso de uso que o estende possui.

Por exemplo, o comportamento associado ao adiar um pagamento é compartilhado como parte de um cenário alternativo de diferentes casos de uso; então ele é identificado e descrito como um caso de uso em si e um relacionamento de extensão é estabelecido entre ele e os demais estendidos.

No diagrama, um relacionamento de extensão é representado como uma seta pontilhada que se origina no caso de uso de extensão em direção ao caso de uso base.

8.12.2.2.3. Inclusão Se há uma sequência de passos comuns e de observação obrigatória em diferentes cenários, então ela deve ser descrita como uma unidade em um caso de uso de inclusão especificamente para esse fim. O caso de uso incluído representa um comportamento obrigatório que o caso de uso que o inclui deve possuir.

Por exemplo, tanto para marcar consulta quanto para pedir remédio há uma série de passos comuns para procurar o registro do paciente.

No diagrama, o relacionamento de inclusão é representado com uma seta pontilhada que se origina no caso de uso base em direção ao caso de uso incluído.

Os relacionamentos de extensão e inclusão só podem ser usados entre casos de uso. A generalização ou especialização pode ser usada entre dois casos de uso ou entre dois atores, mas não entre um ator e um caso de uso.

8.12.3. Especificação dos casos de uso

Sobre o detalhamento do caso de uso, cada um deve ter o seu comportamento descrito em um documento, como parte de um documento, ou em uma ferramenta de gerência de requisitos. A UML não estabelece um padrão para esse fim. Deve-se estabelecer um modelo que melhor atenda às características do contexto em que o método será utilizado.

Independentemente de um modelo em particular, a especificação deve atender às necessidades de informação relacionadas a seguir: ➤

Nome do caso de uso.



Breve descrição com o resumo lógico do comportamento do caso de uso.



Os atores que interagem com o software em análise.



As pré-condições necessárias ao início do caso de uso e as pós-condições esperadas ao seu término.



A sequência de passos que descreve o fluxo principal com o intercâmbio de informação entre o usuário e o software e que descreve os requisitos de armazenamento associados.



Diferentes sequências de passos organizadas em cenários que descrevem fluxos alternativos e de exceção.



A descrição das regras de negócio que se aplicam, ou, preferencialmente, a referência às regras de negócio descritas em um espaço especificamente para esse fim (já que as regras de negócio são potencialmente compartilhadas, pois se aplicam a diferentes passos de diferentes cenários e casos de uso).

8.12.3.1. Cenários

A especificação de um caso de uso é, basicamente, a descrição dos cenários que o compõem. Um cenário corresponde a diferentes passos que se desdobram a partir de um evento que inicia o caso de uso e das condições que afetam como ele deve se comportar.

A descrição de um cenário explora:



como e quando um caso de uso começa; ➤
quando o caso de uso interage com os atores e quais dados eles trocam entre si; ➤
quando o caso de uso referencia ou mantém dados; ➤
como e quando o caso de uso termina.

A descrição de um cenário não aborda aspectos da interface gráfica com o usuário, plataforma de hardware ou software e qualquer requisito não funcional.

O termo “fluxo” também é usado para designar um cenário. Uma diferença entre “fluxo” e “cenário” é que os cenários são subdivididos em dois tipos, enquanto os fluxos são subdivididos em três tipos. São eles: ➤

Principal: descrição que reflete um único cenário para atingir o objetivo do caso de uso, sem qualquer consideração com possíveis falhas. É o “caminho feliz”, curso típico de eventos, fluxo normal, fluxo básico.



Alternativo: cumpre o papel de complementar o cenário principal com fragmentos de fluxos que apresentam o comportamento esperado sob condições alternativas que podem ocorrer no decorrer do cenário principal.



Exceção: no caso de uso do termo “fluxo”, há um tipo de cenário alternativo denominado fluxo de exceção especificamente para descrever o comportamento associado às condições de exceção.

8.12.3.2. Como identificar um caso de uso?

A identificação de um caso de uso está intimamente relacionada às decisões sobre quais tarefas do usuário devem ser transferidas para o software. Cada uma delas deve ser mapeada para um caso de uso e os diferentes cenários

associados descritos. Ao elaborar essa descrição, identificam-se oportunidades de melhorar a qualidade da especificação por meio de novos casos de uso, agora de inclusão ou extensão, conforme o que já foi discutido.

8.12.3.3. Como descrever um caso de uso?

A descrição do caso de uso deve prever os cenários relativos à avaliação das condições que se apliquem; porém, sem descrever as regras de negócio associadas. Isso deve ser especificado à parte. Se uma ferramenta de gerência de requisitos é utilizada, essa separação se torna irrelevante, já que a especificação do caso de uso passa a ser um simples relatório e não o repositório centralizado da informação.

8.12.4. Por que casos de uso?

Com a informação organizada em um modelo funcional baseado em casos de uso, é fornecida informação para trabalhar em três áreas muito importantes nos projetos: ➤

Definição de requisitos: geralmente novos requisitos geram novos casos de uso conforme o sistema vai sendo analisado e modelado.



Comunicação com os clientes: por ser simples, sua compreensão não exige conhecimentos técnicos. O cliente pode entender facilmente tanto o diagrama quanto a especificação, o que facilita a comunicação da equipe técnica com os clientes.



Geração de casos de teste: a junção de todos os cenários para um caso de uso pode sugerir uma bateria de testes para cada cenário.

8.13. Técnica: listas de verificação

O uso de lista de verificação ou *checklist* tem grande apelo como ferramenta de segurança no trabalho. Um caso que ilustra isso surgiu durante a fase final de avaliação das aeronaves especificadas pelo exército dos EUA em 18 de julho de 1934. Três fábricas tinham apresentado aeronaves para testes. Um problema foi encontrado e uma das aeronaves parou de funcionar durante a subida, explodindo em chamas no momento do impacto. Uma investigação foi feita e descobriu-se que a causa do problema foi um erro da pilotagem. O copiloto havia se esquecido de liberar a trava do elevador antes da decolagem. Uma vez no ar, o piloto percebeu o que estava acontecendo e tentou alcançar a maçaneta de bloqueio do elevador, mas já era tarde demais.

Para que problemas como esse não se repetissem, pilotos se uniram e concluíram que era necessária alguma maneira de ter certeza de que tudo foi feito. Quatro listas de verificação foram desenvolvidas: decolagem, voo, antes do pouso e após o desembarque. Os pilotos perceberam que o problema aconteceu devido ao sistema de aviação ser complexo para a memória de um ser humano. As listas de verificação para o piloto e o copiloto garantiram que nada seria esquecido. Com as listas de verificação, um planejamento cuidadoso e rigoroso treinamento, a aeronave conseguiu voar 1.800.000 milhas sem um acidente sequer.

A ideia de lista de verificação do piloto foi bem aceita e mais listas foram desenvolvidas para outros membros da tripulação. Assim, o uso de listas de verificação passou a ser disseminada na aviação.

No contexto da Engenharia de Requisitos, o uso dessas listas nas atividades de verificação e validação é também muito útil.

8.13.1. O que são listas de verificação?

As equipes de projeto ou os responsáveis pelas políticas gerais de qualidade em um nível mais elevado na organização determinam critérios para uso como referência de qualidade no processo de desenvolvimento de software. Eles indicam os elementos críticos e definem as condições que devem se confirmar ao se avaliar uma especificação ou modelo. Os tipos de especificação e modelos, e as condições-alvo associadas, refletem lições aprendidas com o sucesso – ou o fracasso – em projetos passados e com

boas práticas de mercado. O critério de qualidade indica quais as condições para o elemento ser avaliado como correto ou errado, como adequado para ser usado em outras oportunidades ou que requeira maior atenção ainda em sua elaboração.

Por exemplo, um documento de visão é uma especificação que descreve o domínio do problema, as necessidades de negócio, o escopo da solução em termos de seus macroprocessos constituintes e de seu ambiente externo qualificado pelas organizações, pessoas e outros agentes que interagem com o software como parte das responsabilidades associadas ao papel que desempenham. A definição desses elementos deve fazer com que o documento de visão cumpra adequadamente o seu propósito. E essa definição pode ter sido fruto de experiência em projetos internos ou boas práticas consolidadas no mercado. Por exemplo, um elemento crítico nessa especificação é a caracterização do papel que os agentes citados desempenham. E um critério de qualidade para verificar isso é haver, organizada na especificação, informação que descreva o conjunto de habilidades e conhecimentos necessários, indicando também responsabilidades e relacionamentos com outros agentes que desempenham outros papéis.

Uma lista de verificação organiza a referência descrita anteriormente na forma de perguntas ou assertivas encadeadas sequencialmente. Com base nelas, produtos de trabalho individuais podem ser avaliados pela comparação da condição em que se encontram com os critérios de qualidade presentes no modelo de referência.

Cada tipo de produto de trabalho da Engenharia de Requisitos deveria ter uma lista de verificação associada.

8.13.1.1. Como usar listas de verificação?

As listas de verificação são usadas durante revisões técnicas formais.

Tipo de produto de trabalho: **Diagrama de Contexto** Elemento: **requisitos de armazenamento externos sobre um conceito de negócio 02.DT.30 – Verificar se a identificação do conceito de negócio está correta** A identificação de um conceito de negócio sobre o qual se recuperam ou armazenam dados é CORRETA quando: - o elemento identificado observa a definição de conceito de negócio (02.DT.29); - há fundamentos para sua identificação nos documentos com a memória da elicitação que devem permitir identificar as necessidades

de armazenamento do usuário a partir de manifestações espontâneas dos envolvidos, ou como resultado da aplicação de técnicas de elicitação por parte da equipe de desenvolvimento; - há fundamentos nas especificações com os requisitos funcionais (se apropriado, conforme a fase).

02.DT.32 – Verificar se o conceito de negócio é externo à solução proposta Um conceito de negócio é EXTERNO À SOLUÇÃO PROPOSTA quando: - não há requisito funcional para a manutenção de dados sobre o conceito de negócio no âmbito da solução proposta; e - há necessariamente e apenas passos ou regras de negócio em requisitos funcionais que recuperam dados sobre o conceito de negócio. Não se trata especificamente de requisitos funcionais descrevendo como a solução deve responder a um evento externo, nem requisitos funcionais descrevendo o envio de dados para integração com outras soluções.

02.DT.33 – Verificar se o conceito de negócio é necessário O conceito de negócio é NECESSÁRIO quando: - estabelece a capacidade de o negócio precisar de resoluções internas; ou - permite conformidade do negócio a exigências, interfaces ou padrões externos. O que evidencia essas condições é o campo “Necessidades Abordadas (N.XX)” ser preenchido com todos os itens do Quadro de Necessidades e Oportunidades que o justificam e apenas esses itens.

02.DT.34 – Verificar se o Quadro de Requisitos de Armazenamento (Externos) está completo O Quadro de Requisitos de Armazenamento – Externos à Solução Proposta – está COMPLETO quando todos os conceitos de negócios a seguir qualificados estejam relacionados e apenas eles: - CORRETOS (02.DT.30); e - EXTERNOS À SOLUÇÃO PROPOSTA (02.DT.32).

Quadro 8.1. Exemplo completo de política de verificação.

Os tipos de produto de trabalho deveriam estar organizados em modelos para facilitar o desenvolvimento. A Figura 8.20 ilustra um fragmento do modelo que aplica a lista de verificação usada como exemplo. Mas atenção: o uso de modelos como o apresentado, sem o entendimento adequado de seu propósito e do que as informações nele presentes representam, traz mais prejuízos que vantagens.

Quadro de Conceitos de Negócio (Assuntos)

Nome

CN.01

Necessidades de Negócio (N.XX)

Fundamentos (ML.XX)

Necessidade de manutenção em outra aplicação?	Em caso positivo, quais aplicações e OOSS
Não ☐ Sim ☐	

Figura 8.20. Fragmento com modelo descrevendo as necessidades de informação.

8.13.2. Por que listas de verificação?

A motivação para usar listas de verificação é facilitar o controle da qualidade para os produtos de trabalho entregues pelo projeto, mais especificamente: ➤

Garantir que as especificações sejam desenvolvidas e verificadas de maneira uniforme.

☐

Descobrir inconsistências entre diferentes modelos.

☐

Encontrar especificações supérfluas para o atendimento dos requisitos de negócio.

☐

Evitar que especificações incompletas avancem no projeto.

☐

Desenvolver projetos mais gerenciáveis.

As listas de verificação ajudam a identificar o que foi feito e em que qualidade, de forma objetiva. Sem o apoio de uma lista de verificação, o trabalho dependeria da experiência e do critério usado por cada um responsável por verificar, o que pode significar uma verificação não padronizada. Um mesmo artefato verificado de maneira *ad hoc* por diferentes pessoas pode levar a resultados distintos, dependendo de quem

verifica. Com a lista de verificação, até o membro menos experiente da equipe pode executar atividades de verificação.

A lista de verificação deveria ser elaborada aproveitando a experiência dos profissionais mais antigos, com olhar mais criterioso e com vivência em vários problemas que poderiam ser detectados pela lista. À medida que problemas forem detectados no projeto, deve-se revisar a lista de verificação para avaliar se algum item pode ser acrescentado ou modificado, para filtrar previamente problemas similares em futuros projetos.

Logo, a lista de verificação ajuda também a diminuir o potencial de falhas em projetos, economiza tempo nas atividades de verificação e permite o compartilhamento de conhecimento e experiência entre os membros da equipe.

8.14. Técnica: prototipação

Estabelecer a proposta de interface gráfica do software com o usuário é uma responsabilidade da disciplina de Análise e Projeto (arquitetura). Um termo de uso mais abrangente denominado “UX” (*User Experience*) tem sido cada vez mais usado e se consolida como especialização profissional, considerando também a integração com elementos de hardware e serviços. A prototipação é fundamental no desenvolvimento da UX de um produto de software... mas não é esse uso que se discute aqui. O enfoque é para a validação e descoberta de requisitos.

A análise de requisitos é responsável por dar orientação na identificação e definição dos elementos lógicos da interface com o usuário – os campos com as informações que devem ser fornecidas pelos usuários, o que deve ser apresentado ao usuário, a estrutura de ambos quanto a tamanho, tipos e regras de validação ou formatação que compõem esses elementos lógicos.

De maneira similar às decisões sobre questões de arquitetura de alto risco, que devem ser abordadas em paralelo ao desenvolvimento do software, é possível haver paralelismo entre a realização da Engenharia de Requisitos e a UX. Enquanto se detalham os requisitos funcionais, tomam-se decisões sobre a UX.

No primeiro caso, quando se discutem as decisões sobre questões de alto risco, utiliza-se o termo “deve” e, no segundo, sobre a experiência do usuário associada ao detalhamento de cada requisito, se estabelece uma possibilidade. Se essas questões de arquitetura de alto risco não forem abordadas logo no início do projeto, então todo o trabalho subsequente é comprometido, inclusive o da Engenharia de Requisitos. Essas decisões estabelecem limites entre o que será especificado como um requisito funcional ou não funcional durante a análise de requisitos e direciona o desenvolvimento subsequente de diversas especialidades. Por exemplo: ➤

Requisitos: o comportamento relativo ao armazenamento de dados com uma trilha de auditoria (*log*) ser especificado como passos em diferentes especificações funcionais ou estabelecido uma única vez em uma especificação complementar de forma geral depende de decisões quanto à arquitetura de software a ser utilizada e os seus componentes de infraestrutura com serviços comuns compartilhados.



Análise e projeto: a alocação de determinado comportamento especificado nos requisitos em uma unidade de software (programa, classe em Java, pacote PL/SQL etc.) depende de decisões prévias sobre a arquitetura.



Implementação: a linguagem de programação depende dessas decisões preliminares de arquitetura.

Já quando se discutem decisões sobre a experiência do usuário associada a cada requisito funcional, trata-se de uma escolha que depende da estratégia de desenvolvimento. É viável que todos os elementos lógicos da interface com o usuário sejam identificados e descritos para, depois, se decidir quanto à representação da informação como um campo de texto, um ícone, uma árvore, o modo para a entrada e validação de dados, quantos formulários deverão suportar o requisito funcional *etc.* A Engenharia de

Requisitos deve estabelecer os requisitos não funcionais da solução; porém, sem entrar nesse mérito.

Da mesma forma, é viável tomar ambas as decisões – sobre os elementos lógicos e a UX – em um mesmo momento, seja por um único perfil profissional que tenha as habilidades associadas à Engenharia de Requisitos e à UX ou por dois perfis profissionais diferentes trabalhando em conjunto.

Um fato que deve ser considerado é que a volatilidade sobre as decisões de UX é maior que a volatilidade associada às necessidades de informação. Trabalhar desenvolvendo os dois concomitantemente requer bastante disciplina para não negligenciar um deles.

8.14.1. O que é prototipação?

A prototipação é uma técnica que busca simular para o usuário o funcionamento dos seus requisitos antes que o produto final esteja pronto. É um processo iterativo onde se geram versões iniciais de protótipos, e por meio deles o usuário poderá analisar se os seus requisitos estão sendo atendidos e até mesmo descobrir novos requisitos. Também é utilizada para melhorar a experiência do usuário, avaliar opções de *design* e montar uma base para o desenvolvimento final do produto.

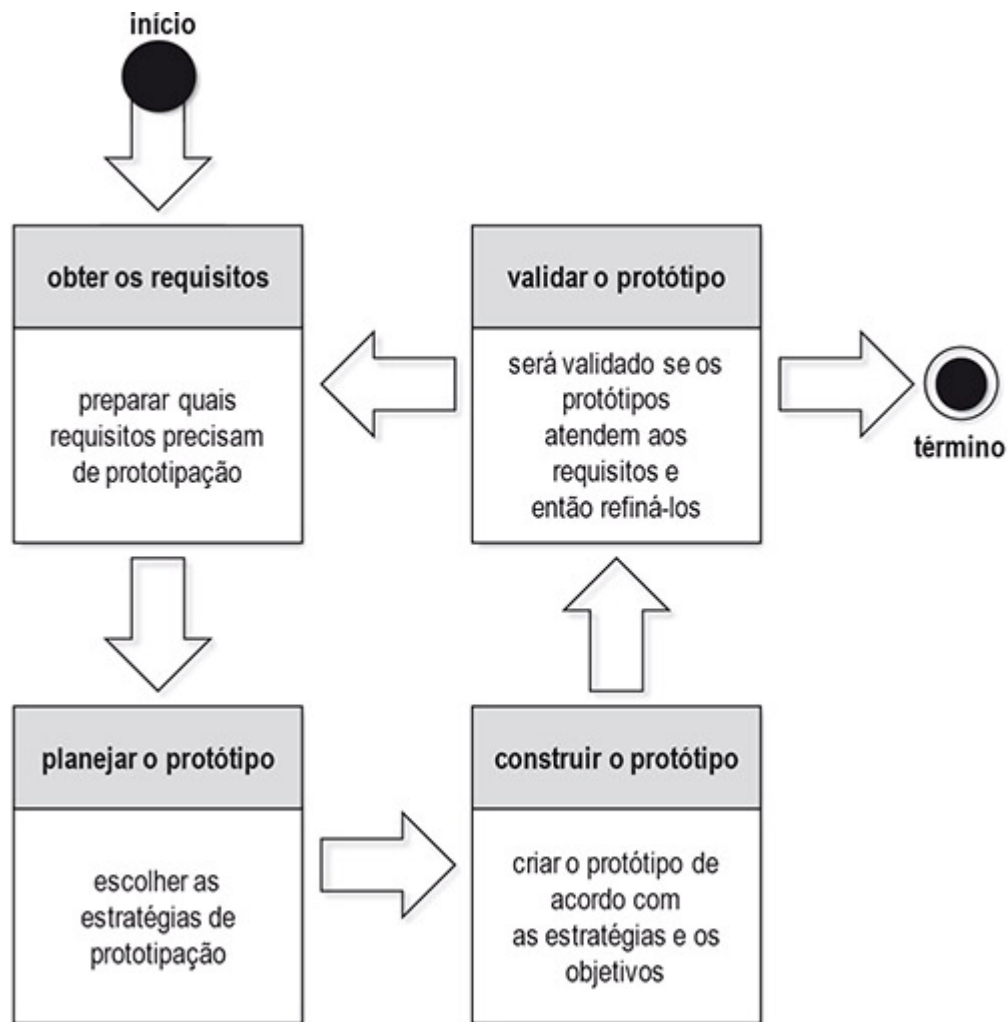


Figura 8.21. Ciclo da prototipação.

8.14.2. Como aplicar a prototipação?

O ciclo de uma prototipação começa na obtenção dos requisitos, passa pelo planejamento de um protótipo, onde são decididas qual técnica e qual abordagem de prototipação utilizar, para então construir um protótipo. Após a construção do protótipo, o usuário e o desenvolvedor irão utilizá-lo para refinar os requisitos. Feito isso, o processo se inicia novamente para uma nova prototipação, até que os requisitos todos estejam maduros para o desenvolvimento.

Diferentes técnicas para a prototipação são apresentadas adiante. A escolha de quais abordagens utilizar deve ser feita com cuidado, condizendo com o objetivo da prototipação, pois estas influenciarão na produtividade e no

custo do seu projeto. Escolher uma abordagem errada para o seu objetivo pode prejudicar o desenvolvimento do seu projeto em vez de ajudá-lo.

Por exemplo, se o objetivo for refinar requisitos funcionais, então não convém elaborar um protótipo com a estética do produto final, incluindo outros elementos além dos elementos lógicos necessários para o objetivo citado. Exceções à regra seriam os casos em que sem esse artifício não se consegue estabelecer a comunicação com uma parte interessada que não tem a capacidade de abstração mínima para entender uma interface apenas com os elementos lógicos, não permitindo o avanço sem a inclusão de outros elementos de usabilidade.

Feita a prototipação, o desenvolvedor e o cliente devem se reunir e avaliar se os protótipos construídos estão de acordo com os requisitos. O *feedback* com essas lacunas é o produto de maior valor da prototipação. Ele servirá de insumo para que os requisitos possam ser mais refinados. Se na primeira sessão de validação do protótipo não forem identificados problemas em requisitos, desconfie e avalie uma nova abordagem na prototipação para chegar ao resultado esperado. Ou avalie se a pessoa designada é a correta para validar o protótipo.

Terminando toda a análise da prototipação, os requisitos devem ser melhorados conforme o *feedback* gerado e, caso necessário, deve-se iniciar o processo de prototipação novamente, de forma que novos problemas sejam encontrados e novas soluções sejam estabelecidas. A cada prototipação, a ideia é que o nível de detalhe e a precisão dos protótipos aumentem até que todas as incertezas dos requisitos sejam solucionadas, para então poder ser construído o produto final.

8.14.2.1. Prototipação de baixa fidelidade × alta fidelidade

A prototipação é classificada como sendo de baixa fidelidade ou de alta fidelidade conforme o nível de aparência do protótipo em comparação ao produto final.

8.14.2.1.1. Prototipação de baixa fidelidade A prototipação de baixa fidelidade é a mais simples em termos de esforço, custo e tempo investido em sua elaboração. É feita em momentos iniciais do projeto. Os protótipos

de baixa fidelidade geralmente são desenhados na forma de esboços e sem muitos detalhes; não há preocupação prioritária com características estéticas. Se houve algo nesse sentido, então os resultados sobre esses elementos não devem ser tratados como uma linha de base para a gerência de mudanças, pois o objetivo está na descoberta dos elementos lógicos da interface com o usuário. Os protótipos criados nesse tipo de prototipação são também denominados *mockups* ou *wireframes*.

Esse tipo de prototipação impede que o usuário confunda o protótipo com um produto final. Esse estilo de protótipo influencia o interesse do usuário; o foco está apenas nas funcionalidades do sistema, entendendo que o protótipo é apenas um esboço que pode ser alterado com facilidade. Isso é positivo, pois não há risco de desvio da atenção do usuário para a parte estética. Uma desvantagem na falta de detalhes é a dificuldade que algumas pessoas têm de abstrair dos atributos adicionais aos elementos lógicos. A Figura 8.22 ilustra um protótipo de baixa fidelidade.

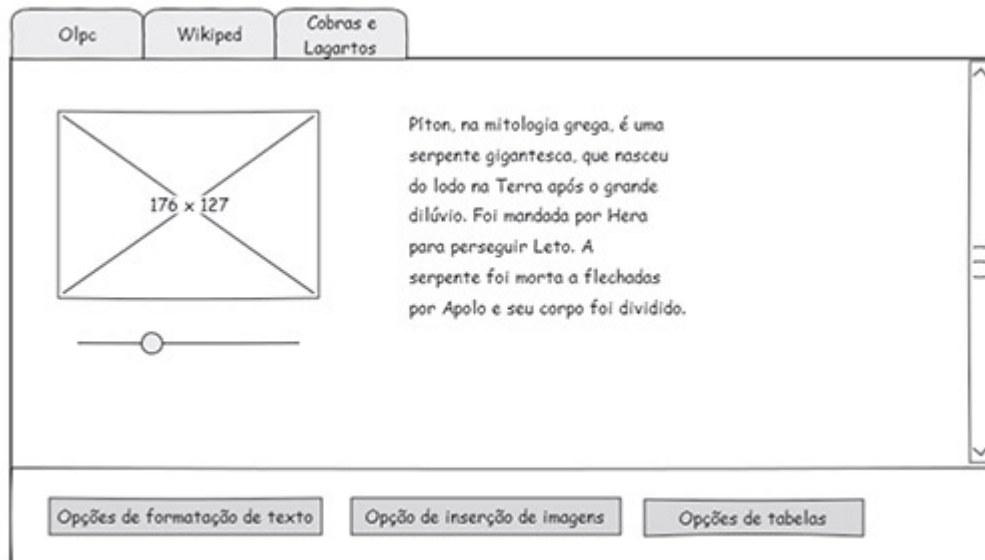


Figura 8.22. Exemplo de protótipo de baixa fidelidade.

8.14.2.1.2. Prototipação de alta fidelidade A prototipação de alta fidelidade apresenta um nível alto de detalhes do ponto de vista da aparência. O usuário deve ser capaz de enxergar do modo mais realista possível como será o produto final. Esses protótipos podem ser construídos usando a própria ferramenta de desenvolvimento de software ou ferramentas de prototipação que permitem simular a experiência de uso do

software sem que haja dados reais disponíveis ou código-fonte programado para esse fim. Essas últimas ferramentas, normalmente, têm o seu comportamento especificado por meio de configuração e, eventualmente, por *scripts* em linguagens de alto nível.

A prototipação de alta fidelidade é ótima para descobrir novos requisitos de usabilidade e outros requisitos não funcionais, porém o usuário pode se distrair e não validar as funcionalidades e seus elementos lógicos. Há o risco de o usuário criar expectativas ao pensar que o protótipo será (ou é) o produto final do software. É uma boa técnica para a prototipação de produtos destinados ao mercado em massa e como ferramenta para estabelecer os requisitos de UX para o produto. Uma desvantagem desta abordagem é que se demora mais para elaborar um protótipo de alta fidelidade do que um de baixa fidelidade.

8.14.2.2. Prototipação horizontal × vertical

Elaborar um protótipo completo para um sistema grande pode levar bastante tempo, e a demora diminui os benefícios da prototipação. Para evitar isso, é comum optar por elaborar protótipos parciais, buscando oferecer ou uma visão ampla (mas rasa) do produto ou uma visão profunda (mas específica) de uma parte do produto. Escolher entre essas duas abordagens requer avaliar o propósito que se deseja alcançar com a prototipação: consolidar o escopo ou detalhar os elementos já presentes no escopo.

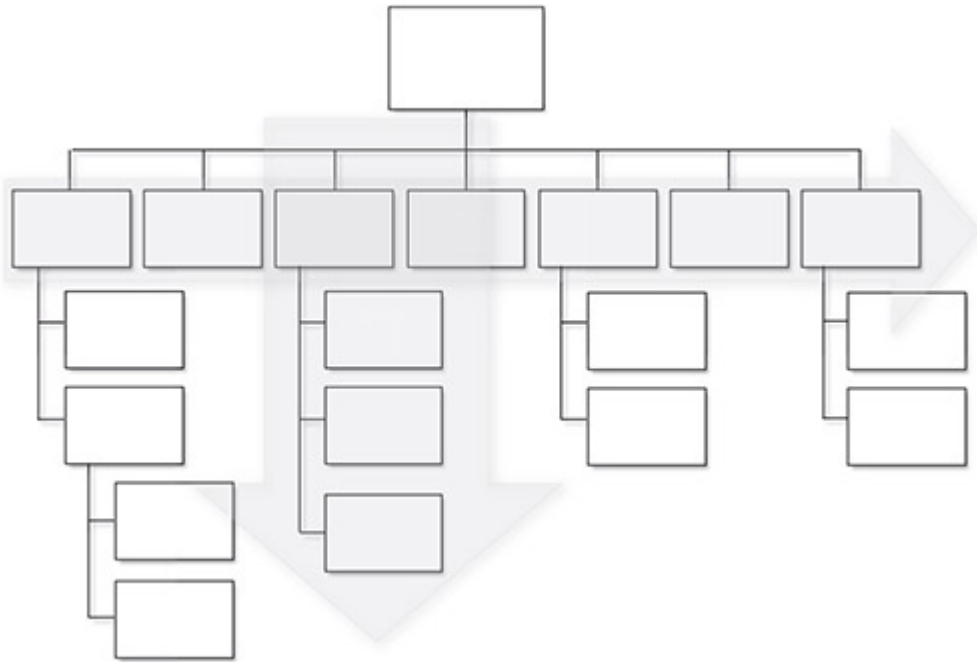


Figura 8.23. Protótipo horizontal x vertical.

8.14.2.2.1. Prototipação horizontal A prototipação horizontal aborda os requisitos do usuário em menor profundidade e busca cobrir de forma ampla várias funcionalidades, porém não se aprofunda no seu funcionamento. Por entrar em menor nível de detalhe, um maior conjunto de funcionalidades pode ser abordado em uma única sessão. Por abordar a abrangência para uma solução, a prototipação horizontal é adequada para as etapas iniciais do projeto. A vantagem desta abordagem é mostrar ao usuário as revelações que o desenvolvimento produziu até o momento sobre o funcionamento geral do sistema.

8.14.2.2.2. Prototipação vertical A prototipação vertical tem por objetivo criar protótipos que exploram as funcionalidades em maior profundidade e permitem explorar um menor número de funcionalidades em uma mesma sessão de prototipação, em comparação aos protótipos mais simplórios de um conjunto maior de funcionalidades na prototipação horizontal.

Esses protótipos são mais adequados em momentos adiantados do desenvolvimento e podem ajudar no refinamento e na avaliação dos requisitos do usuário. A vantagem desta abordagem é demonstrar requisitos específicos e aprofundar o entendimento dos detalhes de cada funcionalidade.

8.14.2.3. Prototipação descartável x evolutiva

Quanto ao ciclo de vida, os protótipos podem ser classificados em descartáveis ou evolutivos.

8.14.2.3.1. Prototipação evolutiva A prototipação evolutiva é a abordagem na qual os protótipos serão evoluídos até se tornarem prontos para compor a versão final do software. São elaborados com a própria ferramenta de desenvolvimento de software. Este é o caso onde o protótipo construído irá sofrer constante evolução até se tornar o resultado final do desenvolvimento. Portanto, ao se desenvolver o protótipo, está também se desenvolvendo o software. Seu ciclo de vida não se encerra com o término das atividades de prototipação. É a abordagem preferida para os projetos desenvolvidos com a metodologia “ágil”. No entanto, a prototipação evolutiva pode demandar a antecipação de questões relativas a disciplinas de *design* e construção, para que se tenha uma arquitetura do software preparada para suportar as sucessivas evoluções do protótipo até o produto final. Isso pode provocar atraso para obter a primeira versão do protótipo. Quando já existe previamente uma arquitetura definida para se usar, então pode ser rápido gerar a primeira versão do protótipo.

8.14.2.3.2. Prototipação descartável O protótipo construído desta forma, após cumprir seu objetivo, será descartado. Ao contrário da prototipação evolutiva, os protótipos não serão itens de configuração para a versão final do software. Os protótipos descartáveis geralmente são feitos com ferramentas próprias para desenho de protótipos. Seu ciclo de vida se encerra depois que se deixa de prototipar no projeto. Em geral são mais rápidos e baratos de produzir.

8.14.2.4. Riscos e cuidados

Ao realizar a prototipação, é importante definir bem com o cliente o objetivo pelo qual o protótipo foi criado. Se esse objetivo não for definido adequadamente, a prototipação pode ter impacto negativo. Seguem alguns cuidados a serem tomados ao realizar a prototipação: ➤

Certifique-se de que os usuários e os desenvolvedores não fiquem muito presos ao protótipo. Se o protótipo estiver bem detalhado e os objetivos com a prototipação não estão claros, então o usuário pode

criar a expectativa de que o sistema já está pronto ou que ficará exatamente igual ao protótipo, quando, dependendo dos objetivos da prototipação, ainda há decisões a serem tomadas sobre a interface de usuário.



Certifique-se de que, quando o usuário aprovar um protótipo de baixa fidelidade para validar os requisitos, os desenvolvedores não considerem o protótipo como o resultado final que descreve toda a UX prevista para o produto. O trabalho para descobrir esses elementos é escopo de atividade desses desenvolvedores, e eles não devem implementar uma tela sem considerar boas práticas de UX só porque o cliente aprovou um protótipo de baixa fidelidade com o objetivo de identificar os elementos lógicos da interface com o usuário.



Certifique-se de que, em uma sessão de prototipação para validar requisitos funcionais, o usuário não se atenha à UX, pois se isso acontecer, tempo de projeto será desperdiçado e serão criadas expectativas erradas. Uma solução é usar um protótipo de baixa fidelidade; assim o cliente tende a acreditar que o sistema final não terá essa aparência rústica ou má usabilidade.



Certifique-se de esclarecer que os tempos instantâneos obtidos na experiência de uso de um protótipo não devem ser comparados aos produtos finais. Como um protótipo é apenas uma ferramenta de simulação de como o sistema ficará quando pronto, não possuindo assim processamento real associado a ele, deve-se ter o cuidado para que o usuário não compare a velocidade no desempenho de um protótipo com o desempenho do produto final.

8.14.2.5. Protótipo x caso de uso

Um protótipo não possui especificação das funcionalidades apresentadas. Sem o conhecimento dos requisitos, quem olhar o protótipo poderá não saber o que as funcionalidades apresentadas fazem. Por isso, a combinação entre protótipo e especificações de casos de uso é muito interessante. O caso de uso documentará o fluxo e o funcionamento das funcionalidades, e estas serão demonstradas pelo protótipo. A junção protótipo e caso de uso também é uma ótima forma de documentar seu sistema para retenção do conhecimento sem a necessidade de aprender a ler alguma linguagem de programação e interpretar o fracionamento dos requisitos em diferentes itens de configuração do software.

8.14.3. Por que prototipação?

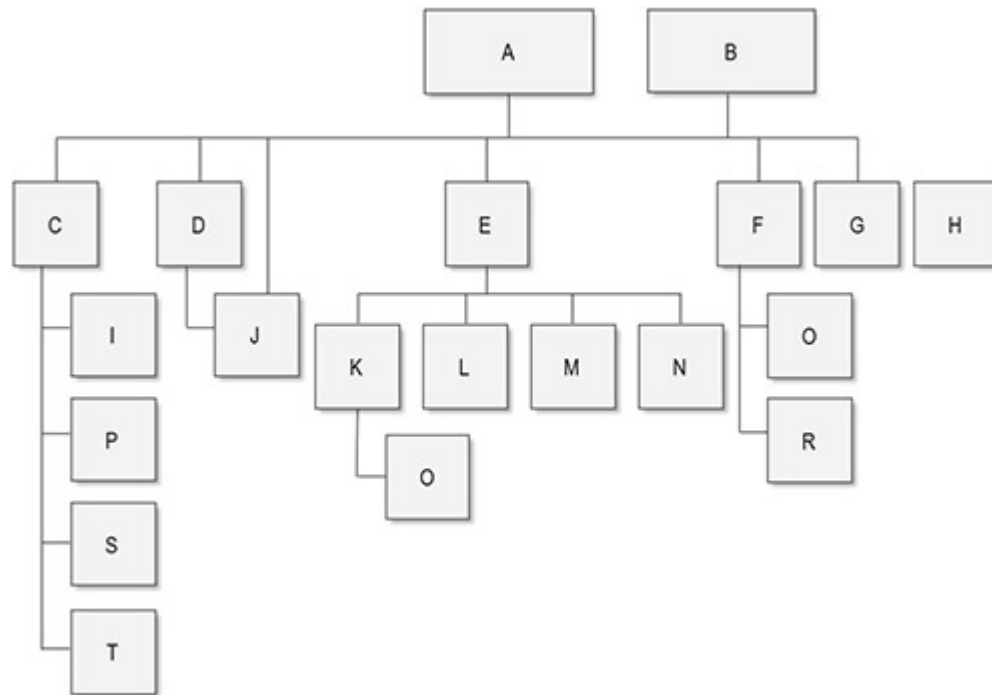
A prototipação tem como objetivo principal diminuir os riscos do projeto, permitindo a descoberta de problemas de requisitos em etapas iniciais que talvez fossem difíceis de identificar com outras técnicas.

A validação de um protótipo expõe mal-entendidos entre os envolvidos no projeto, assim como traz à tona os requisitos implícitos (ou “óbvios”). No primeiro caso, é útil tanto para identificar problemas de comunicação entre partes interessadas e equipe de requisitos quanto para ajudar a solucionar conflitos de visão entre as partes interessadas sobre qual melhor opção de solução para o produto. No segundo caso, dos requisitos implícitos, ao perceber durante a validação do protótipo que este não possui uma característica esperada, mas que não foi solicitada de forma explícita, o usuário irá apontar e reclamar.

Quanto mais cedo se introduz o protótipo no projeto, maior o potencial de benefício que pode ser obtido. Se a prototipação está sendo feita em etapas avançadas do projeto ou se está consumindo muito tempo, é sinal de que algo não está bem e os benefícios potenciais da prototipação podem se perder. Embora seja uma prática comum em muitas empresas, é frequente cometer o “pecado” de deixar para prototipar no projeto de forma tardia.

8.15. Exercícios

1. Qual é o objetivo da Análise de Requisitos?
2. A figura a seguir representa a decomposição funcional de um sistema genérico. Verifique se a decomposição foi feita corretamente. Caso encontre problemas, liste-os.



3. Ao fazer a decomposição dos requisitos em um modelo de processos, foram encontradas algumas atividades sem nenhum requisito alocado. O que isso significa?
4. Utilizando o estudo de caso do Anexo como referência, identifique os seus requisitos de armazenamento (conceitos de negócio). Registre também as dúvidas que surgirem nesse processo. Estas demandarão novas atividades de elicitação. Você pode usar a lista a seguir para ajudá-lo nessa identificação. Procure buscar por conceitos de negócio relacionados a:
 - a) documentos tramitados e cujos dados são informados, transformados, armazenados e distribuídos em seu teor bruto e/ou com informações derivadas a partir deles;
 - b) pessoas representando os diferentes papéis que precisam interagir;
 - c)

eventos que demarcam diferentes pontos no processo de negócio sendo abordado ou diferentes resultados entregues em marcos;

d)

manifestações que provocam a ação e o acompanhamento dessa ação; e)

estruturas organizacionais com suas alçadas e competências; f)

parâmetros operacionais que limitam e estabelecem políticas; g)

estruturas físicas com a sua descrição e qualificação, como prédios, plantas, fábricas, produtos, depósitos, localizações, equipamentos *etc.*

8. Utilizando o estudo de caso do Anexo como referência, elabore um diagrama de casos de uso que represente o escopo deste projeto. Registre também as dúvidas que surgirem nesse processo. Estas demandarão novas atividades de elicitação.
9. Utilizando o estudo de caso do Anexo como referência, especifique duas histórias do usuário. Elabore-as de acordo com o formato sugerido na lição: “como (papel), eu quero (algo) para (me beneficiar)”.